Extraction du code du dossier : C:\dossier tlf\New folder\text exporter
Date d'extraction : 15/08/2026 17:43:07
==========

STRUCTURE DU PROJET
==========
[DIR] text exporter/
???   [PY] compile_po.py (1.96 Ko)
???   [FILE] config.json (839 octets)
???   [TXT] extractor_version.txt (46 octets)
???   [PY] main.py (323 octets)
???   [FILE] recent_folders.json (72 octets)
??? [DIR] locale/
| ??? [DIR] en/
| | ??? [DIR] LC_MESSAGES/
| | | ???   [MO] messages.mo (3.66 Ko)
| | | ???   [PO] messages.po (3.95 Ko)
| ??? [DIR] fr/
| | ??? [DIR] LC_MESSAGES/
| | | ???   [MO] messages.mo (3.96 Ko)
| | | ???   [PO] messages.po (4.25 Ko)
??? [DIR] out/
| ???   [TXT] Site webv1.txt (24.12 Ko)
| ???   [TXT] project - LVv1.txt (20.99 Ko)
| ???   [TXT] project - LVv2.txt (26.21 Ko)
| ???   [TXT] project - LVv3.txt (36.20 Ko)
| ???   [TXT] project - LVv4.txt (39.85 Ko)
| ???   [TXT] text exporterv51.txt (195.74 Ko)
| ???   [TXT] text exporterv52.txt (230.04 Ko)
| ???   [TXT] text exporterv53.txt (229.59 Ko)
| ???   [TXT] text exporterv54.txt (233.82 Ko)
| ???   [TXT] text exporterv55.txt (0 octets)
| ??? [DIR] old_out/
| | ???   [TXT] text exporterv1.txt (21.69 Ko)
| | ???   [TXT] text exporterv2.txt (23.74 Ko)
| | ???   [TXT] text exporterv3.txt (23.37 Ko)
| | ???   [TXT] text exporterv4.txt (25.07 Ko)
| | ???   [TXT] text exporterv5.txt (31.42 Ko)
| | ???   [TXT] text exporterv6.txt (34.24 Ko)
| | ??? [DIR] text exporter/
| | | ???   [TXT] text exporterv1.txt (35.81 Ko)
| | | ???   [TXT] text exporterv10.txt (64.95 Ko)
| | | ???   [TXT] text exporterv11.txt (65.08 Ko)
| | | ???   [TXT] text exporterv12.txt (67.46 Ko)
| | | ???   [TXT] text exporterv13.txt (67.96 Ko)
| | | ???   [TXT] text exporterv14.txt (67.78 Ko)
| | | ???   [TXT] text exporterv15.txt (68.70 Ko)
| | | ???   [TXT] text exporterv16.txt (93.24 Ko)
| | | ???   [TXT] text exporterv17.txt (92.40 Ko)
| | | ???   [TXT] text exporterv18.txt (72.71 Ko)
| | | ???   [TXT] text exporterv19.txt (73.73 Ko)
| | | ???   [TXT] text exporterv2.txt (50.46 Ko)
| | | ???   [TXT] text exporterv20.txt (73.77 Ko)
| | | ???   [TXT] text exporterv21.txt (74.82 Ko)

```
|   |   |   ???   [TXT]  text exporterv22.txt (74.95 Ko)
|   |   |   ???   [TXT]  text exporterv23.txt (75.08 Ko)
|   |   |   ???   [TXT]  text exporterv24.txt (74.24 Ko)
|   |   |   ???   [TXT]  text exporterv25.txt (76.12 Ko)
|   |   |   ???   [TXT]  text exporterv26.txt (81.16 Ko)
|   |   |   ???   [TXT]  text exporterv27.txt (87.17 Ko)
|   |   |   ???   [TXT]  text exporterv28.txt (102.32 Ko)
|   |   |   ???   [TXT]  text exporterv29.txt (105.06 Ko)
|   |   |   ???   [TXT]  text exporterv3.txt (53.03 Ko)
|   |   |   ???   [TXT]  text exporterv30.txt (115.63 Ko)
|   |   |   ???   [TXT]  text exporterv31.txt (116.12 Ko)
|   |   |   ???   [TXT]  text exporterv32.txt (125.40 Ko)
|   |   |   ???   [TXT]  text exporterv33.txt (125.47 Ko)
|   |   |   ???   [TXT]  text exporterv34.txt (126.65 Ko)
|   |   |   ???   [TXT]  text exporterv35.txt (127.32 Ko)
|   |   |   ???   [TXT]  text exporterv36.txt (127.44 Ko)
|   |   |   ???   [TXT]  text exporterv37.txt (136.55 Ko)
|   |   |   ???   [TXT]  text exporterv38.txt (140.98 Ko)
|   |   |   ???   [TXT]  text exporterv39.txt (136.02 Ko)
|   |   |   ???   [TXT]  text exporterv4.txt (57.03 Ko)
|   |   |   ???   [TXT]  text exporterv40.txt (136.43 Ko)
|   |   |   ???   [TXT]  text exporterv41.txt (156.41 Ko)
|   |   |   ???   [TXT]  text exporterv42.txt (156.76 Ko)
|   |   |   ???   [TXT]  text exporterv43.txt (156.81 Ko)
|   |   |   ???   [TXT]  text exporterv44.txt (134.56 Ko)
|   |   |   ???   [TXT]  text exporterv45.txt (135.76 Ko)
|   |   |   ???   [TXT]  text exporterv46.txt (134.03 Ko)
|   |   |   ???   [TXT]  text exporterv47.txt (134.13 Ko)
|   |   |   ???   [TXT]  text exporterv48.txt (134.19 Ko)
|   |   |   ???   [TXT]  text exporterv49.txt (135.66 Ko)
|   |   |   ???   [TXT]  text exporterv5.txt (57.58 Ko)
|   |   |   ???   [TXT]  text exporterv50.txt (137.31 Ko)
|   |   |   ???   [TXT]  text exporterv6.txt (57.63 Ko)
|   |   |   ???   [TXT]  text exporterv7.txt (62.06 Ko)
|   |   |   ???   [TXT]  text exporterv8.txt (62.12 Ko)
|   |   |   ???   [TXT]  text exporterv9.txt (0 octets)
|   |   ???  [DIR]  received/
???  [DIR]  src/
|   |   ???   [PY]  __init__.py (0 octets)
|   |   ???   [PY]  i18n.py (1.56 Ko)
|   |   ???   [PY]  logger.py (553 octets)
|   |   ???   [PY]  utils.py (575 octets)
|   |   ???   [PY]  versioning.py (3.80 Ko)
|   |   ???  [DIR]  __pycache__/
|   |   |   ???   [FILE]  __init__.cpython-311.pyc (164 octets)
|   |   |   ???   [FILE]  __init__.cpython-313.pyc (129 octets)
|   |   |   ???   [FILE]  __init__.cpython-314.pyc (159 octets)
|   |   |   ???   [FILE]  config.cpython-311.pyc (5.36 Ko)
|   |   |   ???   [FILE]  extractor.cpython-313.pyc (13.56 Ko)
|   |   |   ???   [FILE]  extractor.cpython-314.pyc (10.08 Ko)
|   |   |   ???   [FILE]  gui.cpython-313.pyc (13.67 Ko)
|   |   |   ???   [FILE]  gui.cpython-314.pyc (13.31 Ko)
|   |   |   ???   [FILE]  i18n.cpython-311.pyc (2.88 Ko)
|   |   |   ???   [FILE]  logger.cpython-311.pyc (1.05 Ko)
```

```
|   |   ???   [FILE]  utils.cpython-311.pyc (956 octets)
|   |   ???   [FILE]  utils.cpython-313.pyc (465 octets)
|   |   ???   [FILE]  utils.cpython-314.pyc (467 octets)
|   |   ???   [FILE]  versioning.cpython-311.pyc (7.50 Ko)
|   |   ???   [FILE]  versioning.cpython-313.pyc (3.72 Ko)
|   |   ???   [FILE]  versioning.cpython-314.pyc (2.37 Ko)
|   ???  [DIR]  config/
|   |   ???   [PY]  __init__.py (2.20 Ko)
|   |   ???   [PY]  schema.py (3.29 Ko)
|   |   ???  [DIR]  __pycache__/
|   |   |   ???   [FILE]  __init__.cpython-311.pyc (3.71 Ko)
|   |   |   ???   [FILE]  schema.cpython-311.pyc (5.18 Ko)
|   ???  [DIR]  extractor/
|   |   ???   [PY]  __init__.py (0 octets)
|   |   ???   [PY]  content_formatter.py (476 octets)
|   |   ???   [PY]  content_reader.py (2.17 Ko)
|   |   ???   [PY]  export_writer.py (2.83 Ko)
|   |   ???   [PY]  extractor.py (7.49 Ko)
|   |   ???   [PY]  file_discovery.py (1.87 Ko)
|   |   ???   [PY]  statistics_collector.py (2.75 Ko)
|   |   ???   [PY]  stats_writer.py (3.01 Ko)
|   |   ???   [PY]  structure_generator.py (2.88 Ko)
|   |   ???  [DIR]  __pycache__/
|   |   |   ???   [FILE]  __init__.cpython-311.pyc (174 octets)
|   |   |   ???   [FILE]  __init__.cpython-313.pyc (139 octets)
|   |   |   ???   [FILE]  content_formatter.cpython-311.pyc (956 octets)
|   |   |   ???   [FILE]  content_formatter.cpython-313.pyc (852 octets)
|   |   |   ???   [FILE]  content_reader.cpython-311.pyc (3.79 Ko)
|   |   |   ???   [FILE]  export_writer.cpython-311.pyc (3.84 Ko)
|   |   |   ???   [FILE]  extraction_config.cpython-311.pyc (1.89 Ko)
|   |   |   ???   [FILE]  extractor.cpython-311.pyc (8.98 Ko)
|   |   |   ???   [FILE]  extractor.cpython-313.pyc (7.61 Ko)
|   |   |   ???   [FILE]  file_discovery.cpython-311.pyc (3.20 Ko)
|   |   |   ???   [FILE]  file_finder.cpython-311.pyc (2.19 Ko)
|   |   |   ???   [FILE]  file_finder.cpython-313.pyc (2.21 Ko)
|   |   |   ???   [FILE]  file_utils.cpython-311.pyc (620 octets)
|   |   |   ???   [FILE]  statistics_collector.cpython-311.pyc (4.23 Ko)
|   |   |   ???   [FILE]  stats_writer.cpython-311.pyc (4.32 Ko)
|   |   |   ???   [FILE]  stats_writer.cpython-313.pyc (2.46 Ko)
|   |   |   ???   [FILE]  structure_generator.cpython-311.pyc (4.13 Ko)
|   |   |   ???   [FILE]  structure_generator.cpython-313.pyc (3.66 Ko)
|   ???  [DIR]  gui/
|   |   ???   [PY]  __init__.py (669 octets)
|   |   ???   [PY]  extraction_controller.py (208 octets)
|   |   ???   [PY]  extraction_runner.py (4.09 Ko)
|   |   ???   [PY]  file_selector.py (415 octets)
|   |   ???   [PY]  folder_scanner.py (1.49 Ko)
|   |   ???   [PY]  gui.py (2.61 Ko)
|   |   ???   [PY]  recent_files.py (1.71 Ko)
|   |   ???  [DIR]  __pycache__/
|   |   |   ???   [FILE]  __init__.cpython-311.pyc (844 octets)
|   |   |   ???   [FILE]  __init__.cpython-313.pyc (133 octets)
|   |   |   ???   [FILE]  extraction_controller.cpython-311.pyc (319 octets)
|   |   |   ???   [FILE]  extraction_controller.cpython-313.pyc (10.10 Ko)
```

```
|   |   |   ???   [FILE]  extraction_runner.cpython-311.pyc (7.00 Ko)
|   |   |   ???   [FILE]  file_selector.cpython-311.pyc (768 octets)
|   |   |   ???   [FILE]  folder_scanner.cpython-311.pyc (2.02 Ko)
|   |   |   ???   [FILE]  gui.cpython-311.pyc (4.84 Ko)
|   |   |   ???   [FILE]  gui.cpython-313.pyc (3.08 Ko)
|   |   |   ???   [FILE]  recent_files.cpython-311.pyc (3.28 Ko)
|   |   |   ???   [FILE]  selection_dialog.cpython-311.pyc (17.54 Ko)
|   |   |   ???   [FILE]  ui_builder.cpython-313.pyc (7.51 Ko)
|   |   |   ???   [FILE]  version_manager.cpython-311.pyc (450 octets)
|   |   ???  [DIR]  controller/
|   |   |   ???   [PY]  __init__.py (505 octets)
|   |   |   ???   [PY]  base_controller.py (3.37 Ko)
|   |   |   ???   [PY]  extraction_controller.py (4.36 Ko)
|   |   |   ???   [PY]  folder_controller.py (2.16 Ko)
|   |   |   ???   [PY]  main_controller.py (5.21 Ko)
|   |   |   ???   [PY]  navigation_controller.py (2.19 Ko)
|   |   |   ???   [PY]  server_controller.py (1.80 Ko)
|   |   |   ???  [DIR]  __pycache__/
|   |   |   |   ???   [FILE]  __init__.cpython-311.pyc (693 octets)
|   |   |   |   ???   [FILE]  base_controller.cpython-311.pyc (6.29 Ko)
|   |   |   |   ???   [FILE]  extraction_controller.cpython-311.pyc (8.58 Ko)
|   |   |   |   ???   [FILE]  folder_controller.cpython-311.pyc (4.33 Ko)
|   |   |   |   ???   [FILE]  main_controller.cpython-311.pyc (9.20 Ko)
|   |   |   |   ???   [FILE]  navigation_controller.cpython-311.pyc (3.74 Ko)
|   |   |   |   ???   [FILE]  server_controller.cpython-311.pyc (3.53 Ko)
|   |   ???  [DIR]  network_center/
|   |   |   ???   [PY]  __init__.py (114 octets)
|   |   |   ???   [PY]  dialog.py (2.73 Ko)
|   |   |   ???   [PY]  log_tab.py (2.86 Ko)
|   |   |   ???   [PY]  received_tab.py (5.95 Ko)
|   |   |   ???   [PY]  send_tab.py (9.20 Ko)
|   |   |   ???   [PY]  status_tab.py (4.80 Ko)
|   |   |   ???  [DIR]  __pycache__/
|   |   |   |   ???   [FILE]  __init__.cpython-311.pyc (273 octets)
|   |   |   |   ???   [FILE]  dialog.cpython-311.pyc (5.42 Ko)
|   |   |   |   ???   [FILE]  log_tab.cpython-311.pyc (6.18 Ko)
|   |   |   |   ???   [FILE]  received_tab.cpython-311.pyc (12.96 Ko)
|   |   |   |   ???   [FILE]  send_tab.cpython-311.pyc (19.75 Ko)
|   |   |   |   ???   [FILE]  status_tab.cpython-311.pyc (9.15 Ko)
|   |   ???  [DIR]  selection/
|   |   |   ???   [PY]  __init__.py (101 octets)
|   |   |   ???   [PY]  dialog.py (5.73 Ko)
|   |   |   ???   [PY]  search.py (1.24 Ko)
|   |   |   ???   [PY]  toolbar.py (2.38 Ko)
|   |   |   ???   [PY]  tree_builder.py (3.12 Ko)
|   |   |   ???   [PY]  tree_controller.py (5.12 Ko)
|   |   |   ???   [PY]  utils.py (1.06 Ko)
|   |   |   ???  [DIR]  __pycache__/
|   |   |   |   ???   [FILE]  __init__.cpython-311.pyc (264 octets)
|   |   |   |   ???   [FILE]  dialog.cpython-311.pyc (10.47 Ko)
|   |   |   |   ???   [FILE]  search.cpython-311.pyc (2.80 Ko)
|   |   |   |   ???   [FILE]  toolbar.cpython-311.pyc (3.98 Ko)
|   |   |   |   ???   [FILE]  tree_builder.cpython-311.pyc (5.00 Ko)
|   |   |   |   ???   [FILE]  tree_controller.cpython-311.pyc (8.05 Ko)
```

```
|   |   |   |   ???   [FILE]  utils.cpython-311.pyc (2.24 Ko)
|   |   ???  [DIR]  ui_builder/
|   |   |   ???   [PY]  __init__.py (0 octets)
|   |   |   ???   [PY]  menus.py (7.44 Ko)
|   |   |   ???   [PY]  ui_builder.py (1.68 Ko)
|   |   |   ???   [PY]  widgets.py (3.63 Ko)
|   |   |   ???  [DIR]  __pycache__/
|   |   |   |   ???   [FILE]  __init__.cpython-311.pyc (179 octets)
|   |   |   |   ???   [FILE]  __init__.cpython-313.pyc (144 octets)
|   |   |   |   ???   [FILE]  menus.cpython-311.pyc (12.15 Ko)
|   |   |   |   ???   [FILE]  menus.cpython-313.pyc (3.02 Ko)
|   |   |   |   ???   [FILE]  ui_builder.cpython-311.pyc (2.76 Ko)
|   |   |   |   ???   [FILE]  ui_builder.cpython-313.pyc (1.54 Ko)
|   |   |   |   ???   [FILE]  widgets.cpython-311.pyc (6.33 Ko)
|   |   |   |   ???   [FILE]  widgets.cpython-313.pyc (4.76 Ko)
|   |   ???  [DIR]  version_explorer/
|   |   |   ???   [PY]  __init__.py (120 octets)
|   |   |   ???   [PY]  dialog.py (11.08 Ko)
|   |   |   ???   [PY]  preview_pane.py (2.47 Ko)
|   |   |   ???   [PY]  project_panel.py (1.77 Ko)
|   |   |   ???   [PY]  toolbar.py (1.70 Ko)
|   |   |   ???   [PY]  utils.py (2.15 Ko)
|   |   |   ???   [PY]  version_tree.py (5.39 Ko)
|   |   |   ???  [DIR]  __pycache__/
|   |   |   |   ???   [FILE]  __init__.cpython-311.pyc (277 octets)
|   |   |   |   ???   [FILE]  dialog.cpython-311.pyc (22.04 Ko)
|   |   |   |   ???   [FILE]  preview_pane.cpython-311.pyc (4.42 Ko)
|   |   |   |   ???   [FILE]  project_panel.cpython-311.pyc (3.65 Ko)
|   |   |   |   ???   [FILE]  toolbar.cpython-311.pyc (3.30 Ko)
|   |   |   |   ???   [FILE]  utils.cpython-311.pyc (3.64 Ko)
|   |   |   |   ???   [FILE]  version_tree.cpython-311.pyc (9.14 Ko)
|   ???  [DIR]  network/
|   |   ???   [PY]  __init__.py (0 octets)
|   |   ???   [PY]  discovery.py (3.44 Ko)
|   |   ???   [PY]  server.py (6.87 Ko)
|   |   ???   [PY]  utils.py (357 octets)
|   |   ???  [DIR]  __pycache__/
|   |   |   ???   [FILE]  __init__.cpython-311.pyc (172 octets)
|   |   |   ???   [FILE]  discovery.cpython-311.pyc (7.03 Ko)
|   |   |   ???   [FILE]  server.cpython-311.pyc (12.08 Ko)
|   |   |   ???   [FILE]  transfer_dialog.cpython-311.pyc (16.69 Ko)
|   |   |   ???   [FILE]  utils.cpython-311.pyc (1013 octets)
|   ???  [DIR]  services/
|   |   ???   [PY]  __init__.py (26 octets)
|   |   ???   [PY]  extraction_service.py (3.60 Ko)
|   |   ???   [PY]  pdf_service.py (6.40 Ko)
|   |   ???   [PY]  version_service.py (8.09 Ko)
|   |   ???  [DIR]  __pycache__/
|   |   |   ???   [FILE]  __init__.cpython-311.pyc (173 octets)
|   |   |   ???   [FILE]  extraction_service.cpython-311.pyc (5.01 Ko)
|   |   |   ???   [FILE]  pdf_service.cpython-311.pyc (7.71 Ko)
|   |   |   ???   [FILE]  version_service.cpython-311.pyc (10.86 Ko)

================================================================================
```

--- FIN DE LA STRUCTURE ---

==
FICHIER Python : compile_po.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\compile_po.py
----
```
# compile_po.py
import polib
from pathlib import Path

def ensure_locale_structure():
    """Crée les dossiers et fichiers .po s'ils n'existent pas."""
    locale_dir = Path("locale")
    languages = ["fr", "en"]

    for lang in languages:
        po_dir = locale_dir / lang / "LC_MESSAGES"
        po_dir.mkdir(parents=True, exist_ok=True)
        po_file = po_dir / "messages.po"

        if not po_file.exists():
            # Créer un fichier .po minimal avec en-tête
            po_content = f'''msgid ""
msgstr ""
"Project-Id-Version: PythonCodeExtractor 1.0\\n"
"POT-Creation-Date: 2026-08-13 11:00+0200\\n"
"PO-Revision-Date: 2026-08-13 11:00+0200\\n"
"Last-Translator: \\n"
"Language-Team: \\n"
"Language: {lang}\\n"
"MIME-Version: 1.0\\n"
"Content-Type: text/plain; charset=UTF-8\\n"
"Content-Transfer-Encoding: 8bit\\n"
'''
            with open(po_file, 'w', encoding='utf-8') as f:
                f.write(po_content)
            print(f"[OK]  Fichier {po_file} créé.")

def compile_all_po():
    """Compile tous les fichiers .po en .mo."""
    locale_dir = Path("locale")
    if not locale_dir.exists():
        print("[ERR]  Le dossier 'locale' n'existe pas. Création...")
        ensure_locale_structure()

    for lang_dir in locale_dir.iterdir():
        if lang_dir.is_dir():
            po_file = lang_dir / "LC_MESSAGES" / "messages.po"
            mo_file = lang_dir / "LC_MESSAGES" / "messages.mo"
            if po_file.exists():
                try:
```

```
                    po = polib.pofile(str(po_file))
                    po.save_as_mofile(str(mo_file))
                    print(f"[OK]  {mo_file} généré avec succès")
                except Exception as e:
                    print(f"[ERR]  Erreur pour {po_file} : {e}")
            else:
                print(f"[WARN]   {po_file} manquant, ignoré.")

if __name__ == "__main__":
    ensure_locale_structure()
    compile_all_po()
    print("[OK]  Compilation terminée.")

--- FIN DU FICHIER ---

================================================================================
FICHIER JSON : config.json
Type: JSON
Chemin complet: C:\dossier tlf\New folder\text exporter\config.json
--------------------------------------------------------------------------------
{
  "output_dir": "out",
  "received_subdir": "received",
  "archive_subdir": "old_out",
  "version_file": "extractor_version.txt",
  "language": "fr",
  "network": {
    "server_host": "0.0.0.0",
    "server_port": 50000,
    "discovery_port": 50001,
    "broadcast_msg": "PYEXTRACTOR_DISCOVER",
    "reply_msg": "PYEXTRACTOR_HERE"
  },
  "extraction": {
    "include_json": true,
    "include_subdirs": true,
    "show_file_paths": true,
    "include_structure": true,
    "include_txt": false,
    "include_po": true,
    "include_mo": true,
    "include_html": true,
    "include_css": true,
    "include_js": true,
    "ignore_init": false,
    "include_statistics": true,
    "include_file_metadata": false
  },
  "gui": {
    "window_width": 700,
    "window_height": 600,
    "log_height": 12
  }
}
```

--- FIN DU FICHIER ---

==
FICHIER Python : main.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\main.py
----
```
# main.py
import tkinter as tk
from src.gui.gui import PythonCodeExtractor
from src.i18n import setup_i18n

def main():
    # Initialiser la traduction avant de créer l'interface
    setup_i18n()
    root = tk.Tk()
    app = PythonCodeExtractor(root)
    root.mainloop()

if __name__ == "__main__":
    main()
```

--- FIN DU FICHIER ---

==
FICHIER JSON : recent_folders.json
Type: JSON
Chemin complet: C:\dossier tlf\New folder\text exporter\recent_folders.json
----
```
{
  "folders": [
    "C:/dossier tlf/New folder/text exporter"
  ]
}
```

--- FIN DU FICHIER ---

==
FICHIER Traduction compilée (MO) : locale\en\LC_MESSAGES\messages.mo
Type: Traduction compilée (MO)
Chemin complet: C:\dossier tlf\New folder\text exporter\locale\en\LC_MESSAGES\messages.mo
----
```
// Fichier binaire (.mo) encodé en base64
3hIElQAAAAA1AAAAHAAAAMQBAAAAAAAAbAMAAAAAAABsAwAAHAAAAG0DAAAqAAAAigMAABYAAAC1AwAAFwAAAMwDAAAUAAAA5AMAABUAAAD5AwAAGAAAAA
cyAtLS0ACi0tLSBFeHRyYWN0aW9uIHRlcm1pbsOpZSBhdmVjIHN1Y2PDqHMgLS0tACAgLSBGaWNoaWVycyBKU09OIDoge30AICAtIEZpY2hpZXJzIEpTT0
YW5jaWVucyBmaWNoaWVycyBzZXJvbnQgYXJjaGl2w6lzIGRhbnMgb3V0L29sZF9vdXQvLgBOb21icmUgZGUgZmljaGllcnMgZXh0cmFpdHMgOiB7fQoAUE
ICAtIFBPIGZpbGVzOiB7fQAgIC0gUE8gZmlsZXM6IHt9CgAgIC0gUHl0aG9uIGZpbGVzOiB7fQAgIC0gUHl0aG9uIGZpbGVzOiB7fQoAICAtIFRYVCBmaW
ZWFzZSBzZWxlY3QgYSBmb2xkZXIgZmlyc3QAU2VuZCBmYWlsZWQAU2VuZCBmYWlsZWQgKHJlZnVzZWQpAFNlbmQgZmFpbGVkICh0aW1lb3V0KQBTZW5kIC
```

--- FIN DU FICHIER ---

==
FICHIER Traduction (PO) : locale\en\LC_MESSAGES\messages.po

Type: Traduction (PO)
Chemin complet: C:\dossier tlf\New folder\text exporter\locale\en\LC_MESSAGES\messages.po
---
msgid ""
msgstr ""
"Project-Id-Version: PythonCodeExtractor 1.0\n"
"POT-Creation-Date: 2026-08-13 11:00+0200\n"
"PO-Revision-Date: 2026-08-13 12:00+0200\n"
"Last-Translator: \n"
"Language-Team: \n"
"Language: en\n"
"MIME-Version: 1.0\n"
"Content-Type: text/plain; charset=UTF-8\n"
"Content-Transfer-Encoding: 8bit\n"

msgid "Extracteur de code Python/JSON/TXT"
msgstr "Python/JSON/TXT Code Extractor"

msgid "Sélectionner un dossier contenant des fichiers Python/JSON/TXT/PO"
msgstr "Select a folder containing Python/JSON/TXT/PO files"

msgid "Dossier sélectionné : {}"
msgstr "Selected folder: {}"

msgid "Fichiers trouvés : {}"
msgstr "Files found: {}"

msgid " - Fichiers Python : {}"
msgstr " - Python files: {}"

msgid " - Fichiers JSON : {}"
msgstr " - JSON files: {}"

msgid " - Fichiers TXT : {}"
msgstr " - TXT files: {}"

msgid " - Fichiers PO : {}"
msgstr " - PO files: {}"

msgid "Attention"
msgstr "Warning"

msgid "Veuillez d'abord sélectionner un dossier"
msgstr "Please select a folder first"

msgid "Extraction en cours..."
msgstr "Extracting..."

msgid "\n--- Extraction en cours ---"
msgstr "\n--- Extraction in progress ---"

msgid "Extraction échouée"
msgstr "Extraction failed"

```
msgid "Erreur"
msgstr "Error"

msgid "Échec de l'extraction (voir journal)"
msgstr "Extraction failed (see log)"

msgid "Extraction terminée"
msgstr "Extraction complete"

msgid "\n--- Extraction terminée avec succès ---"
msgstr "\n--- Extraction completed successfully ---"

msgid "Fichier créé : {}"
msgstr "File created: {}"

msgid "Emplacement : {}"
msgstr "Location: {}"

msgid "Fichiers extraits : {} (Python: {}, JSON: {}, TXT: {}, PO: {})"
msgstr "Files extracted: {} (Python: {}, JSON: {}, TXT: {}, PO: {})"

msgid "Extraction terminée avec succès !\n\n"
msgstr "Extraction completed successfully!\n\n"

msgid "Fichier créé : {}\n"
msgstr "File created: {}\n"

msgid "Nombre de fichiers extraits : {}\n"
msgstr "Number of files extracted: {}\n"

msgid "  - Fichiers Python : {}\n"
msgstr "  - Python files: {}\n"

msgid "  - Fichiers JSON : {}\n"
msgstr "  - JSON files: {}\n"

msgid "  - Fichiers TXT : {}\n"
msgstr "  - TXT files: {}\n"

msgid "  - Fichiers PO : {}\n"
msgstr "  - PO files: {}\n"

msgid "Emplacement : {}"
msgstr "Location: {}"

msgid "Succès"
msgstr "Success"

msgid "Une erreur est survenue : {}"
msgstr "An error occurred: {}"

msgid "Confirmation"
msgstr "Confirmation"
```

```
msgid "Réinitialiser toutes les versions et l'historique ?\n"
msgstr "Reset all versions and history?\n"

msgid "Les anciens fichiers seront archivés dans out/old_out/."
msgstr "Old files will be archived in out/old_out/."

msgid "--- {} ---"
msgstr "--- {} ---"

msgid "Échec du nettoyage : {}"
msgstr "Cleanup failed: {}"

msgid "Impossible d'ouvrir la fenêtre de transfert : {}"
msgstr "Cannot open transfer dialog: {}"

msgid "Envoyer un fichier à un autre PC"
msgstr "Send file to another PC"

msgid "Fichiers disponibles dans out/ :"
msgstr "Files available in out/:"

msgid "PC disponibles sur le réseau :"
msgstr "PCs available on network:"

msgid "Envoyer le fichier"
msgstr "Send file"

msgid "Rafraîchir la liste"
msgstr "Refresh list"

msgid "Scanner le réseau"
msgstr "Scan network"

msgid "Prêt"
msgstr "Ready"

msgid "Erreur de lecture des fichiers"
msgstr "Error reading files"

msgid "Fichier envoyé à {} ({})"
msgstr "File sent to {} ({})"

msgid "Envoi réussi"
msgstr "Send successful"

msgid "Le serveur distant ne répond pas (timeout)"
msgstr "Remote server not responding (timeout)"

msgid "Échec de l'envoi (timeout)"
msgstr "Send failed (timeout)"

msgid "Le serveur distant a refusé la connexion"
msgstr "Remote server refused connection"
```

```
msgid "Échec de l'envoi (refus)"
msgstr "Send failed (refused)"

msgid "Échec de l'envoi : {}"
msgstr "Send failed: {}"

msgid "Échec de l'envoi"
msgstr "Send failed"

--- FIN DU FICHIER ---

================================================================================
FICHIER Traduction compilée (MO) : locale\fr\LC_MESSAGES\messages.mo
Type: Traduction compilée (MO)
Chemin complet: C:\dossier tlf\New folder\text exporter\locale\fr\LC_MESSAGES\messages.mo
--------------------------------------------------------------------------------
// Fichier binaire (.mo) encodé en base64
3hIElQAAAAA1AAAAHAAAAMQBAAAAAAAAbAMAAAAAAABsAwAAHAAAAG0DAAAqAAAAigMAABYAAAC1AwAAFwAAAMwDAAAUAAAA5AMAABUAAAD5AwAAGAAAAA
cyAtLS0ACi0tLSBFeHRyYWN0aW9uIHRlcm1pbsOpZSBhdmVjIHN1Y2PDqHMgLS0tACAgLSBGaWNoaWVycyBKU09OIDoge30AICAtIEZpY2hpZXJzIEpTT0
YW5jaWVucyBmaWNoaWVycyBzZXJvbnQgYXJjaGl2lzIGRhbnMgb3V0L29sZF9vdXQvLgBOb21icmUgZGUgZmljaGllcnMgZXh0cmFpdHMgOiB7fQoAUE
IHt9CgAgIC0gRmljaGllcnMgUE8gOiB7fQAgIC0gRmljaGllcnMgUE8gOiB7fQoAICAtIEZpY2hpZXJzIFB5dGhvbiA6IHt9ACAgLSBGaWNoaWVycyBQeX
IHt9CgBQQyBkaXNwb25pYmxlcyBzdXIgbGUgcsOpc2VhdSA6AFByw6p0AFJhZnJhw65jaGlyIGxhIGxpc3RlAFLDqWluaXRpYWxpc2VyIHRvdXRlcyBsZX

--- FIN DU FICHIER ---

================================================================================
FICHIER Traduction (PO) : locale\fr\LC_MESSAGES\messages.po
Type: Traduction (PO)
Chemin complet: C:\dossier tlf\New folder\text exporter\locale\fr\LC_MESSAGES\messages.po
--------------------------------------------------------------------------------
msgid ""
msgstr ""
"Project-Id-Version: PythonCodeExtractor 1.0\n"
"POT-Creation-Date: 2026-08-13 11:00+0200\n"
"PO-Revision-Date: 2026-08-13 12:00+0200\n"
"Last-Translator: \n"
"Language-Team: \n"
"Language: fr\n"
"MIME-Version: 1.0\n"
"Content-Type: text/plain; charset=UTF-8\n"
"Content-Transfer-Encoding: 8bit\n"

msgid "Extracteur de code Python/JSON/TXT"
msgstr "Extracteur de code Python/JSON/TXT"

msgid "Sélectionner un dossier contenant des fichiers Python/JSON/TXT/PO"
msgstr "Sélectionner un dossier contenant des fichiers Python/JSON/TXT/PO"

msgid "Dossier sélectionné : {}"
msgstr "Dossier sélectionné : {}"

msgid "Fichiers trouvés : {}"
msgstr "Fichiers trouvés : {}"
```

```
msgid "  - Fichiers Python : {}"
msgstr "  - Fichiers Python : {}"

msgid "  - Fichiers JSON : {}"
msgstr "  - Fichiers JSON : {}"

msgid "  - Fichiers TXT : {}"
msgstr "  - Fichiers TXT : {}"

msgid "  - Fichiers PO : {}"
msgstr "  - Fichiers PO : {}"

msgid "Attention"
msgstr "Attention"

msgid "Veuillez d'abord sélectionner un dossier"
msgstr "Veuillez d'abord sélectionner un dossier"

msgid "Extraction en cours..."
msgstr "Extraction en cours..."

msgid "\n--- Extraction en cours ---"
msgstr "\n--- Extraction en cours ---"

msgid "Extraction échouée"
msgstr "Extraction échouée"

msgid "Erreur"
msgstr "Erreur"

msgid "Échec de l'extraction (voir journal)"
msgstr "Échec de l'extraction (voir journal)"

msgid "Extraction terminée"
msgstr "Extraction terminée"

msgid "\n--- Extraction terminée avec succès ---"
msgstr "\n--- Extraction terminée avec succès ---"

msgid "Fichier créé : {}"
msgstr "Fichier créé : {}"

msgid "Emplacement : {}"
msgstr "Emplacement : {}"

msgid "Fichiers extraits : {} (Python: {}, JSON: {}, TXT: {}, PO: {})"
msgstr "Fichiers extraits : {} (Python: {}, JSON: {}, TXT: {}, PO: {})"

msgid "Extraction terminée avec succès !\n\n"
msgstr "Extraction terminée avec succès !\n\n"

msgid "Fichier créé : {}\n"
msgstr "Fichier créé : {}\n"
```

```
msgid "Nombre de fichiers extraits : {}\n"
msgstr "Nombre de fichiers extraits : {}\n"

msgid "  - Fichiers Python : {}\n"
msgstr "  - Fichiers Python : {}\n"

msgid "  - Fichiers JSON : {}\n"
msgstr "  - Fichiers JSON : {}\n"

msgid "  - Fichiers TXT : {}\n"
msgstr "  - Fichiers TXT : {}\n"

msgid "  - Fichiers PO : {}\n"
msgstr "  - Fichiers PO : {}\n"

msgid "Emplacement : {}"
msgstr "Emplacement : {}"

msgid "Succès"
msgstr "Succès"

msgid "Une erreur est survenue : {}"
msgstr "Une erreur est survenue : {}"

msgid "Confirmation"
msgstr "Confirmation"

msgid "Réinitialiser toutes les versions et l'historique ?\n"
msgstr "Réinitialiser toutes les versions et l'historique ?\n"

msgid "Les anciens fichiers seront archivés dans out/old_out/."
msgstr "Les anciens fichiers seront archivés dans out/old_out/."

msgid "--- {} ---"
msgstr "--- {} ---"

msgid "Échec du nettoyage : {}"
msgstr "Échec du nettoyage : {}"

msgid "Impossible d'ouvrir la fenêtre de transfert : {}"
msgstr "Impossible d'ouvrir la fenêtre de transfert : {}"

msgid "Envoyer un fichier à un autre PC"
msgstr "Envoyer un fichier à un autre PC"

msgid "Fichiers disponibles dans out/ :"
msgstr "Fichiers disponibles dans out/ :"

msgid "PC disponibles sur le réseau :"
msgstr "PC disponibles sur le réseau :"

msgid "Envoyer le fichier"
msgstr "Envoyer le fichier"
```

```
msgid "Rafraîchir la liste"
msgstr "Rafraîchir la liste"

msgid "Scanner le réseau"
msgstr "Scanner le réseau"

msgid "Prêt"
msgstr "Prêt"

msgid "Erreur de lecture des fichiers"
msgstr "Erreur de lecture des fichiers"

msgid "Fichier envoyé à {} ({})"
msgstr "Fichier envoyé à {} ({})"

msgid "Envoi réussi"
msgstr "Envoi réussi"

msgid "Le serveur distant ne répond pas (timeout)"
msgstr "Le serveur distant ne répond pas (délai dépassé)"

msgid "Échec de l'envoi (timeout)"
msgstr "Échec de l'envoi (délai dépassé)"

msgid "Le serveur distant a refusé la connexion"
msgstr "Le serveur distant a refusé la connexion"

msgid "Échec de l'envoi (refus)"
msgstr "Échec de l'envoi (refus)"

msgid "Échec de l'envoi : {}"
msgstr "Échec de l'envoi : {}"

msgid "Échec de l'envoi"
msgstr "Échec de l'envoi"

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\i18n.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\i18n.py
--------------------------------------------------------------------------------
# src/i18n.py
import gettext
from pathlib import Path
from src.config import config
from src.logger import setup_logger

logger = setup_logger(__name__)

class Translator:
    """Wrapper mutable pour la fonction de traduction."""
    def __init__(self):
```

```
        self._func = gettext.gettext   # fallback par défaut

    def __call__(self, message):
        return self._func(message)

    def set_translation(self, func):
        """Change la fonction de traduction sous-jacente."""
        self._func = func

# Instance globale unique
_ = Translator()

def setup_i18n():
    """Configure la traduction en fonction de la langue définie dans config."""
    lang = config.get('language', 'fr')
    locale_dir = Path(__file__).parent.parent / "locale"

    try:
        translation = gettext.translation(
            'messages',
            localedir=locale_dir,
            languages=[lang],
            fallback=True
        )
        # Installe pour les appels builtins (gettext.install) si nécessaire
        translation.install()
        # Met à jour notre wrapper
        _.set_translation(translation.gettext)
        logger.info(f"Traduction chargée pour la langue '{lang}'")
    except Exception as e:
        logger.warning(
            f"Traduction non disponible pour '{lang}', utilisation du fallback : {e}"
        )
        # Installe le fallback
        gettext.install('messages', names=('ngettext',))
        _.set_translation(gettext.gettext)

    return _

def get_current_language():
    return config.get('language', 'fr')

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\logger.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\logger.py
--------------------------------------------------------------------------------
# src/logger.py
import logging
import sys

def setup_logger(name=None, level=logging.INFO):
```

```
    """Configure et retourne un logger avec un format standard."""
    logger = logging.getLogger(name or __name__)
    if not logger.handlers:
        handler = logging.StreamHandler(sys.stdout)
        handler.setFormatter(logging.Formatter(
            '%(asctime)s - %(name)s - %(levelname)s - %(message)s',
            datefmt='%Y-%m-%d %H:%M:%S'
        ))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\utils.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\utils.py
--------------------------------------------------------------------------------
# src/utils.py
from datetime import datetime

def get_current_date():
    """Retourne la date actuelle formatée"""
    return datetime.now().strftime("%d/%m/%Y %H:%M:%S")

def human_size(size_bytes):
    """
    Retourne une taille lisible en octets, Ko ou Mo.
    Utilisée pour l'affichage des métadonnées (structure, statistiques, GUI).
    """
    if size_bytes < 1024:
        return f"{size_bytes} octets"
    elif size_bytes < 1024 * 1024:
        return f"{size_bytes / 1024:.2f} Ko"
    else:
        return f"{size_bytes / (1024 * 1024):.2f} Mo"

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\versioning.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\versioning.py
--------------------------------------------------------------------------------
# versioning.py
from pathlib import Path
import threading
from src.logger import setup_logger

logger = setup_logger(__name__)

class VersionManager:
    def __init__(self, version_file="extractor_version.txt"):
```

```
    self.version_file = Path(version_file)
    self._lock = threading.Lock()
    self.mapping = self.load_mapping()

def load_mapping(self):
    mapping = {}
    if self.version_file.exists():
        try:
            with open(self.version_file, "r", encoding="utf-8") as f:
                for line in f:
                    line = line.strip()
                    if line and ":" in line:
                        folder, ver = line.split(":", 1)
                        mapping[folder] = int(ver)
        except (IOError, OSError, ValueError) as e:
            logger.error(f"Erreur lors du chargement du fichier de versions : {e}")
            mapping = {}
    return mapping

def save_mapping(self):
    # Sauvegarde atomique via fichier temporaire
    try:
        tmp_file = self.version_file.with_suffix(".tmp")
        with open(tmp_file, "w", encoding="utf-8") as f:
            for folder, ver in sorted(self.mapping.items()):
                f.write(f"{folder}:{ver}\n")
        tmp_file.replace(self.version_file)
    except (IOError, OSError) as e:
        logger.error(f"Erreur lors de la sauvegarde du fichier de versions : {e}")

def get_next_version(self, folder_name, output_dir="."):
    with self._lock:
        output_dir = Path(output_dir)
        last_version = self.mapping.get(folder_name, 0)
        version = last_version + 1
        while (output_dir / f"{folder_name}v{version}.txt").exists():
            version += 1
        return version

def use_version(self, folder_name, version):
    # Tout sous le même verrou pour éviter toute race
    with self._lock:
        self.mapping[folder_name] = version
        # Sauvegarde immédiate avant de libérer le verrou
        self._save_mapping_locked()   # méthode interne sans verrou

def _save_mapping_locked(self):
    """Appelée uniquement lorsqu'on détient déjà le verrou."""
    try:
        tmp_file = self.version_file.with_suffix(".tmp")
        with open(tmp_file, "w", encoding="utf-8") as f:
            for folder, ver in sorted(self.mapping.items()):
                f.write(f"{folder}:{ver}\n")
        tmp_file.replace(self.version_file)
```

```
        except (IOError, OSError) as e:
            logger.error(f"Erreur lors de la sauvegarde du fichier de versions : {e}")

    # On garde save_mapping pour d'autres usages éventuels, mais on le réécrit
    # pour qu'il utilise le verrou (appel externe)
    def save_mapping(self):
        with self._lock:
            self._save_mapping_locked()

    def reset(self):
        with self._lock:
            self.mapping = {}
            if self.version_file.exists():
                try:
                    self.version_file.unlink()
                    logger.info("Fichier de versions supprimé.")
                except (IOError, OSError) as e:
                    logger.error(f"Erreur lors de la suppression du fichier de versions : {e}")

    def reset_project(self, folder_name):
        """Réinitialise le compteur pour un projet spécifique."""
        with self._lock:
            if folder_name in self.mapping:
                del self.mapping[folder_name]
                self._save_mapping_locked()
                logger.info(f"Compteur réinitialisé pour {folder_name}")

    # Anciennes méthodes conservées pour compatibilité
    def load_version(self):
        return 1

    def save_version(self, version):
        pass

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\__init__.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\__init__.py
--------------------------------------------------------------------------------



--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\config\schema.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\config\schema.py
--------------------------------------------------------------------------------
# src/config/__init__.py
# src/config/schema.py
from pydantic import BaseModel, Field
```

```
class NetworkConfig(BaseModel):
    """Configuration réseau pour le serveur et la découverte."""
    server_host: str = Field(default="0.0.0.0", description="Hôte d'écoute du serveur")
    server_port: int = Field(default=50000, ge=1024, le=65535, description="Port du serveur TCP")
    discovery_port: int = Field(default=50001, ge=1024, le=65535, description="Port UDP pour la découverte réseau")
    broadcast_msg: str = Field(default="PYEXTRACTOR_DISCOVER", description="Message de découverte broadcast")
    reply_msg: str = Field(default="PYEXTRACTOR_HERE", description="Message de réponse à la découverte")


class ExtractionOptions(BaseModel):
    """Options de l'extraction."""
    include_json: bool = Field(default=True, description="Inclure les fichiers .json")
    include_subdirs: bool = Field(default=True, description="Parcourir les sous-dossiers")
    show_file_paths: bool = Field(default=True, description="Afficher les chemins complets des fichiers")
    include_structure: bool = Field(default=True, description="Inclure la structure du projet dans l'export")
    include_txt: bool = Field(default=False, description="Inclure les fichiers .txt")
    include_po: bool = Field(default=False, description="Inclure les fichiers .po (traductions)")
    include_mo: bool = Field(default=False, description="Inclure les fichiers .mo (compilés)")
    include_html: bool = Field(default=True, description="Inclure les fichiers .html et .htm")
    include_css: bool = Field(default=True, description="Inclure les fichiers .css")
    include_js: bool = Field(default=True, description="Inclure les fichiers .js")
    ignore_init: bool = Field(default=False, description="Ignorer les fichiers __init__.py")
    include_statistics: bool = Field(default=True, description="Inclure les statistiques dans l'export")
    include_file_metadata: bool = Field(default=False, description="Inclure les métadonnées par fichier (taille, ligne


class GuiConfig(BaseModel):
    """Configuration de l'interface graphique."""
    window_width: int = Field(default=700, ge=400, le=1920, description="Largeur de la fenêtre")
    window_height: int = Field(default=600, ge=300, le=1080, description="Hauteur de la fenêtre")
    log_height: int = Field(default=12, ge=4, le=30, description="Hauteur du journal en lignes")


class AppConfig(BaseModel):
    """Configuration principale de l'application."""
    output_dir: str = Field(default="out", description="Dossier de sortie des exports")
    received_subdir: str = Field(default="received", description="Sous-dossier pour les fichiers reçus")
    archive_subdir: str = Field(default="old_out", description="Sous-dossier pour l'archive des anciennes versions")
    version_file: str = Field(default="extractor_version.txt", description="Nom du fichier de version")
    language: str = Field(default="fr", description="Langue de l'interface (fr ou en)")

    network: NetworkConfig = Field(default_factory=NetworkConfig, description="Configuration réseau")
    extraction: ExtractionOptions = Field(default_factory=ExtractionOptions, description="Options d'extraction")
    gui: GuiConfig = Field(default_factory=GuiConfig, description="Configuration de l'interface")

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\config\__init__.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\config\__init__.py
--------------------------------------------------------------------------------
# src/config/__init__.py
```

```
import json
from pathlib import Path
from .schema import AppConfig, ExtractionOptions, NetworkConfig, GuiConfig

__all__ = ['AppConfig', 'ExtractionOptions', 'NetworkConfig', 'GuiConfig', 'config']

# Chemin vers config.json à la racine du projet
CONFIG_PATH = Path(__file__).parent.parent.parent / "config.json"


class Config:
    """Chargeur de configuration centralisé."""

    _instance = None
    _config: AppConfig = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._load()
        return cls._instance

    @classmethod
    def _load(cls):
        """Charge la configuration depuis config.json ou utilise les valeurs par défaut."""
        default = AppConfig()
        if CONFIG_PATH.exists():
            try:
                with open(CONFIG_PATH, 'r', encoding='utf-8') as f:
                    data = json.load(f)
                cls._config = AppConfig(**data)
                print(f"[OK]  Configuration chargée depuis {CONFIG_PATH}")
            except Exception as e:
                print(f"[ERR]  Erreur de chargement de config.json : {e}")
                print("   -> Utilisation des valeurs par défaut")
                cls._config = default
        else:
            print(f"??  Fichier {CONFIG_PATH} non trouvé, utilisation des valeurs par défaut")
            cls._config = default

    def get(self, key: str, default=None):
        """
        Récupère une valeur par chemin pointé (ex: 'network.server_port').

        Args:
            key: Chemin pointé (ex: 'extraction.include_json')
            default: Valeur par défaut si la clé n'existe pas

        Returns:
            La valeur configurée ou default
        """
        keys = key.split('.')
        value = self._config
        for k in keys:
```

```
            if hasattr(value, k):
                value = getattr(value, k)
            else:
                return default
        return value

    def get_all(self) -> AppConfig:
        """Retourne l'objet AppConfig complet."""
        return self._config


# Instance unique (singleton)
config = Config()

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\extractor\content_formatter.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\extractor\content_formatter.py
--------------------------------------------------------------------------------
# src/extractor/content_formatter.py
import json

def format_json_content(content, file_path=None):
    """Essaie de formater joliment le contenu JSON.
       En cas d'échec, retourne le contenu brut précédé d'un commentaire d'erreur.
    """
    try:
        json_data = json.loads(content)
        return json.dumps(json_data, indent=2, ensure_ascii=False)
    except json.JSONDecodeError as e:
        return f"// ERREUR: JSON invalide - {str(e)}\n{content}"

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\extractor\content_reader.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\extractor\content_reader.py
--------------------------------------------------------------------------------
# src/extractor/content_reader.py
import base64
from pathlib import Path
from typing import Tuple

from src.logger import setup_logger

logger = setup_logger(__name__)


class ContentReader:
    """Responsable de la lecture du contenu des fichiers avec détection d'encodage."""
```

```
    @staticmethod
    def read_file_content(full_path: Path, ext: str) -> Tuple[str, int, int, bool]:
        """
        Lit le contenu d'un fichier selon son extension.
        Retourne (content, num_lines, file_size, read_ok).
        """
        try:
            if ext == '.mo':
                with open(full_path, 'rb') as f:
                    raw = f.read()
                content = base64.b64encode(raw).decode('ascii')
                num_lines = len(content.splitlines())
                file_size = len(raw)
                read_ok = True
            else:
                content = ContentReader._read_text_with_fallback(full_path)
                if ext == '.json':
                    from .content_formatter import format_json_content
                    content = format_json_content(content, full_path)
                num_lines = len(content.splitlines())
                file_size = full_path.stat().st_size
                read_ok = True
        except Exception as e:
            logger.error(f"Erreur de lecture du fichier {full_path} : {e}")
            content = f"// ERREUR: Impossible de lire le fichier : {str(e)}\n"
            num_lines = 0
            file_size = 0
            read_ok = False
        return content, num_lines, file_size, read_ok

    @staticmethod
    def _read_text_with_fallback(file_path: Path) -> str:
        """Tente plusieurs encodages pour lire un fichier texte."""
        encodings = ['utf-8', 'latin-1', 'cp1252', 'iso-8859-15', 'utf-16']
        for enc in encodings:
            try:
                with open(file_path, 'r', encoding=enc) as f:
                    return f.read()
            except (UnicodeDecodeError, UnicodeError):
                continue
        # Dernier recours : latin-1 (ne lève jamais d'exception)
        with open(file_path, 'r', encoding='latin-1') as f:
            return f.read()

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\extractor\export_writer.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\extractor\export_writer.py
--------------------------------------------------------------------------------
# src/extractor/export_writer.py
from pathlib import Path
from src.utils import get_current_date
```

```
from src.utils import human_size
from .stats_writer import write_statistics_section

def write_header(output_file, folder):
    output_file.write(f"Extraction du code du dossier : {folder}\n")
    output_file.write(f"Date d'extraction : {get_current_date()}\n")
    output_file.write("=" * 80 + "\n\n")

def write_file_section(output_file, full_path, rel_path, ext, file_type,
                       content, num_lines, file_size, read_ok,
                       show_file_paths, include_file_metadata):
    full_path = Path(full_path)
    rel_path = Path(rel_path)
    output_file.write(f"\n{'=' * 80}\n")
    output_file.write(f"FICHIER {file_type} : ")
    if show_file_paths:
        output_file.write(f"{rel_path}\n")
    else:
        output_file.write(f"{full_path.name}\n")
    output_file.write(f"Type: {file_type}\n")
    output_file.write(f"Chemin complet: {full_path}\n")

    if include_file_metadata:
        parent_dir = rel_path.parent if show_file_paths else full_path.parent
        if not parent_dir or str(parent_dir) == '.':
            parent_dir = "."
        output_file.write(f"Dossier parent: {parent_dir}\n")
        output_file.write(f"Taille: {human_size(file_size)}\n")
        output_file.write(f"Nombre de lignes: {num_lines}\n")

    output_file.write("-" * 80 + "\n")
    if ext == '.mo' and read_ok:
        output_file.write("// Fichier binaire (.mo) encodé en base64\n")
        output_file.write(content)
        if not content.endswith('\n'):
            output_file.write("\n")
    else:
        output_file.write(content)
        if not content.endswith('\n'):
            output_file.write("\n")
    output_file.write("\n--- FIN DU FICHIER ---\n")

def write_statistics(output_file, folder, include_subdirs,
                     py_count, json_count, txt_count, po_count, mo_count,
                     html_count, css_count, js_count,
                     total_lines_py, total_lines_json, total_lines_txt,
                     total_lines_po, total_lines_mo,
                     total_lines_html, total_lines_css, total_lines_js,
                     total_size,
                     include_statistics):
    if include_statistics:
        output_file.write("\n--- FIN DES FICHIERS ---\n\n")
        write_statistics_section(output_file, folder, include_subdirs,
                                 py_count, json_count, txt_count, po_count, mo_count,
```

```
                                html_count, css_count, js_count,
                                total_lines_py, total_lines_json, total_lines_txt,
                                total_lines_po, total_lines_mo,
                                total_lines_html, total_lines_css, total_lines_js,
                                total_size)

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\extractor\extractor.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\extractor\extractor.py
--------------------------------------------------------------------------------
# src/extractor/extractor.py
from pathlib import Path
from typing import Optional, List, Callable
from src.config import ExtractionOptions   # <-- Changement ici
from .file_discovery import FileDiscoveryService
from .content_reader import ContentReader
from .statistics_collector import StatisticsCollector
from .export_writer import write_header, write_file_section, write_statistics
from .structure_generator import generate_project_structure
from src.logger import setup_logger

logger = setup_logger(__name__)

class CodeExtractor:
    def __init__(self, options: Optional[ExtractionOptions] = None, **overrides):
        # Si aucune option n'est fournie, on prend les valeurs par défaut
        if options is None:
            base = ExtractionOptions()
        else:
            base = options

        # Appliquer les overrides (provenant de l'interface utilisateur)
        if overrides:
            base_dict = base.dict()
            base_dict.update(overrides)
            self.options = ExtractionOptions(**base_dict)
        else:
            self.options = base

        self.discovery = FileDiscoveryService(self.options)
        self.reader = ContentReader()
        self.stats = StatisticsCollector()

    def find_files(self, folder: str):
        """Retourne la liste des fichiers trouvés (compatibilité)."""
        return self.discovery.find_files(folder)

    def generate_project_structure(self, folder: str) -> str:
        """Génère la structure du projet (compatibilité)."""
        return generate_project_structure(
            folder,
```

```
        include_json=self.options.include_json,
        include_subdirs=self.options.include_subdirs,
        include_txt=self.options.include_txt,
        include_po=self.options.include_po,
        include_mo=self.options.include_mo,
        include_html=self.options.include_html,
        include_css=self.options.include_css,
        include_js=self.options.include_js,
        ignore_init=self.options.ignore_init
    )

def extract_all(
    self,
    folder: str,
    output_filename: str,
    progress_callback: Optional[Callable[[int, int], None]] = None,
    log_callback: Optional[Callable[[str], None]] = None,
    selected_files: Optional[List[str]] = None
) -> tuple:
    """
    Point d'entrée principal.
    Retourne (success, py_count, json_count, txt_count, po_count, mo_count,
              html_count, css_count, js_count)
    """
    # 1) Découverte des fichiers
    all_files = self.discovery.find_files(folder)
    if not all_files:
        msg = "Aucun fichier trouvé"
        if log_callback:
            log_callback(msg)
        logger.warning(msg)
        return False, 0, 0, 0, 0, 0, 0, 0, 0

    # 2) Filtrage si selected_files est fourni
    if selected_files is not None:
        selected_set = set(Path(f).as_posix() for f in selected_files)
        files = [(full, rel, ext) for full, rel, ext in all_files
                 if rel.as_posix() in selected_set]
        if not files:
            msg = "Aucun fichier sélectionné ne correspond aux fichiers trouvés"
            if log_callback:
                log_callback(msg)
            logger.warning(msg)
            return False, 0, 0, 0, 0, 0, 0, 0, 0
    else:
        files = all_files

    total_files = len(files)
    output_path = Path(output_filename)
    output_path.parent.mkdir(parents=True, exist_ok=True)

    # 3) Réinitialisation des stats
    self.stats.reset()
```

```
try:
    with open(output_path, 'w', encoding='utf-8') as out_file:
        # En-tête
        write_header(out_file, folder)

        # Structure du projet (optionnelle)
        if self.options.include_structure:
            structure = self.generate_project_structure(folder)
            out_file.write(structure)
            out_file.write("\n--- FIN DE LA STRUCTURE ---\n\n")
            if log_callback:
                log_callback("[OK]  Structure du projet générée")
            logger.info("Structure du projet générée")

        # Traitement des fichiers
        for i, (full_path, rel_path, ext) in enumerate(files):
            if progress_callback:
                progress_callback(i + 1, total_files)

            file_type = self._get_file_type(ext)
            content, num_lines, file_size, read_ok = self.reader.read_file_content(full_path, ext)

            # Écrire la section du fichier
            write_file_section(
                out_file, full_path, rel_path, ext, file_type,
                content, num_lines, file_size, read_ok,
                self.options.show_file_paths,
                self.options.include_file_metadata
            )

            # Mise à jour des statistiques
            self.stats.count_file(ext, num_lines, file_size, read_ok)

            if log_callback:
                log_callback(f"[OK]  {rel_path} extrait" if read_ok else f"[ERR]  Erreur sur {rel_path}")

        # Statistiques finales (optionnelles)
        if self.options.include_statistics:
            out_file.write("\n--- FIN DES FICHIERS ---\n\n")
            # On récupère les compteurs
            (py_count, json_count, txt_count, po_count, mo_count,
             html_count, css_count, js_count) = self.stats.get_counts()
            (total_lines_py, total_lines_json, total_lines_txt,
             total_lines_po, total_lines_mo,
             total_lines_html, total_lines_css, total_lines_js) = self.stats.get_line_counts()
            total_size = self.stats.get_total_size()

            from .stats_writer import write_statistics_section
            write_statistics_section(
                out_file, folder, self.options.include_subdirs,
                py_count, json_count, txt_count, po_count, mo_count,
                html_count, css_count, js_count,
                total_lines_py, total_lines_json, total_lines_txt,
                total_lines_po, total_lines_mo,
```

```
                        total_lines_html, total_lines_css, total_lines_js,
                        total_size
                    )

                logger.info(f"Extraction terminée avec succès : {output_filename}")
                (py_count, json_count, txt_count, po_count, mo_count,
                 html_count, css_count, js_count) = self.stats.get_counts()
                return True, py_count, json_count, txt_count, po_count, mo_count, html_count, css_count, js_count

        except Exception as e:
            logger.error(f"Erreur lors de l'extraction globale : {e}")
            if log_callback:
                log_callback(f"Erreur critique : {e}")
            return False, 0, 0, 0, 0, 0, 0, 0

    @staticmethod
    def _get_file_type(ext: str) -> str:
        return {
            '.py': "Python",
            '.json': "JSON",
            '.po': "Traduction (PO)",
            '.mo': "Traduction compilée (MO)",
            '.html': "HTML",
            '.htm': "HTML",
            '.css': "CSS",
            '.js': "JavaScript"
        }.get(ext, "Texte")

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\extractor\file_discovery.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\extractor\file_discovery.py
--------------------------------------------------------------------------------
# src/extractor/file_discovery.py
from pathlib import Path
from typing import List, Tuple
from src.config import ExtractionOptions   # <-- Import ajouté

class FileDiscoveryService:
    def __init__(self, options: ExtractionOptions):
        self.options = options

    def find_files(self, folder: str) -> List[Tuple[Path, Path, str]]:
        folder_path = Path(folder)
        extensions = self._build_extension_list()
        files = []
        if self.options.include_subdirs:
            for file_path in folder_path.rglob('*'):
                if file_path.is_file() and file_path.suffix in extensions:
                    if self.options.ignore_init and file_path.name == '__init__.py':
                        continue
                    rel_path = file_path.relative_to(folder_path)
```

```
                    files.append((file_path, rel_path, file_path.suffix))
        else:
            for file_path in folder_path.iterdir():
                if file_path.is_file() and file_path.suffix in extensions:
                    if self.options.ignore_init and file_path.name == '__init__.py':
                        continue
                    files.append((file_path, Path(file_path.name), file_path.suffix))
        return files

    def _build_extension_list(self) -> List[str]:
        extensions = ['.py']
        if self.options.include_json:
            extensions.append('.json')
        if self.options.include_txt:
            extensions.append('.txt')
        if self.options.include_po:
            extensions.append('.po')
        if self.options.include_mo:
            extensions.append('.mo')
        if self.options.include_html:
            extensions.extend(['.html', '.htm'])
        if self.options.include_css:
            extensions.append('.css')
        if self.options.include_js:
            extensions.append('.js')
        return extensions

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\extractor\statistics_collector.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\extractor\statistics_collector.py
--------------------------------------------------------------------------------
# src/extractor/statistics_collector.py
from typing import Dict, List, Tuple


class StatisticsCollector:
    """Agrège les statistiques d'extraction."""

    def __init__(self):
        self.reset()

    def reset(self):
        self.py_count = 0
        self.json_count = 0
        self.txt_count = 0
        self.po_count = 0
        self.mo_count = 0
        self.html_count = 0
        self.css_count = 0
        self.js_count = 0
```

```
        self.total_lines_py = 0
        self.total_lines_json = 0
        self.total_lines_txt = 0
        self.total_lines_po = 0
        self.total_lines_mo = 0
        self.total_lines_html = 0
        self.total_lines_css = 0
        self.total_lines_js = 0
        self.total_size = 0

        self.processed_files = 0  # nombre de fichiers traités (même en erreur)

    def count_file(self, ext: str, num_lines: int, file_size: int, read_ok: bool):
        """Met à jour les compteurs pour un fichier donné."""
        if not read_ok:
            self.processed_files += 1
            return

        self.processed_files += 1
        self.total_size += file_size

        if ext == '.py':
            self.py_count += 1
            self.total_lines_py += num_lines
        elif ext == '.json':
            self.json_count += 1
            self.total_lines_json += num_lines
        elif ext == '.po':
            self.po_count += 1
            self.total_lines_po += num_lines
        elif ext == '.mo':
            self.mo_count += 1
            self.total_lines_mo += num_lines
        elif ext in ('.html', '.htm'):
            self.html_count += 1
            self.total_lines_html += num_lines
        elif ext == '.css':
            self.css_count += 1
            self.total_lines_css += num_lines
        elif ext == '.js':
            self.js_count += 1
            self.total_lines_js += num_lines
        else:
            self.txt_count += 1
            self.total_lines_txt += num_lines

    def get_counts(self):
        """Retourne un tuple (py_count, json_count, txt_count, po_count, mo_count, html_count, css_count, js_count)."
        return (self.py_count, self.json_count, self.txt_count, self.po_count, self.mo_count,
                self.html_count, self.css_count, self.js_count)

    def get_line_counts(self):
        """Retourne un tuple (total_lines_py, total_lines_json, total_lines_txt, total_lines_po, total_lines_mo, total
        return (self.total_lines_py, self.total_lines_json, self.total_lines_txt,
```

```
                self.total_lines_po, self.total_lines_mo,
                self.total_lines_html, self.total_lines_css, self.total_lines_js)

    def get_total_size(self):
        return self.total_size

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\extractor\stats_writer.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\extractor\stats_writer.py
--------------------------------------------------------------------------------
# src/extractor/stats_writer.py
from pathlib import Path

def write_statistics_section(output_file, folder, include_subdirs,
                             py_count, json_count, txt_count, po_count, mo_count,
                             html_count, css_count, js_count,
                             total_lines_py, total_lines_json, total_lines_txt,
                             total_lines_po, total_lines_mo,
                             total_lines_html, total_lines_css, total_lines_js,
                             total_size):
    folder = Path(folder)
    num_dirs = 0
    num_packages = 0
    if include_subdirs:
        for root in folder.rglob('*'):
            if root.is_dir() and root != folder:
                num_dirs += 1
                if (root / '__init__.py').exists():
                    num_packages += 1
    else:
        if (folder / '__init__.py').exists():
            num_packages = 1
        num_dirs = 0

    output_file.write("\n" + "=" * 80 + "\n")
    output_file.write("STATISTIQUES\n")
    output_file.write("=" * 80 + "\n")
    output_file.write(f"Nombre de dossiers          : {num_dirs}\n")
    output_file.write(f"Dossiers 'Python package'    : {num_packages}\n")
    output_file.write(f"Fichiers Python (.py)        : {py_count}\n")
    output_file.write(f"Fichiers JSON (.json)        : {json_count}\n")
    output_file.write(f"Fichiers Texte (.txt)        : {txt_count}\n")
    output_file.write(f"Fichiers PO (.po)            : {po_count}\n")
    output_file.write(f"Fichiers MO (.mo)            : {mo_count}\n")
    output_file.write(f"Fichiers HTML (.html/.htm)   : {html_count}\n")
    output_file.write(f"Fichiers CSS (.css)          : {css_count}\n")
    output_file.write(f"Fichiers JavaScript (.js)    : {js_count}\n")
    output_file.write(f"Nombre total de fichiers     : {py_count + json_count + txt_count + po_count + mo_count + html
    output_file.write(f"Nombre de lignes (Python)    : {total_lines_py}\n")
    output_file.write(f"Nombre de lignes (JSON)      : {total_lines_json}\n")
    output_file.write(f"Nombre de lignes (Texte)     : {total_lines_txt}\n")
```

```
    output_file.write(f"Nombre de lignes (PO)        : {total_lines_po}\n")
    output_file.write(f"Nombre de lignes (MO)        : {total_lines_mo}\n")
    output_file.write(f"Nombre de lignes (HTML)      : {total_lines_html}\n")
    output_file.write(f"Nombre de lignes (CSS)       : {total_lines_css}\n")
    output_file.write(f"Nombre de lignes (JS)        : {total_lines_js}\n")
    output_file.write(f"Nombre total de lignes       : {total_lines_py + total_lines_json + total_lines_txt + total_li

    if total_size < 1024:
        size_str = f"{total_size} octets"
    elif total_size < 1024 * 1024:
        size_str = f"{total_size / 1024:.2f} Ko"
    else:
        size_str = f"{total_size / (1024 * 1024):.2f} Mo"
    output_file.write(f"Volume total extrait         : {size_str}\n")

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\extractor\structure_generator.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\extractor\structure_generator.py
--------------------------------------------------------------------------------
# src/extractor/structure_generator.py
from pathlib import Path
from src.utils import human_size

def generate_project_structure(folder, include_json=True, include_subdirs=True,
                               include_txt=False, include_po=False, include_mo=False,
                               include_html=True, include_css=True, include_js=True,
                               ignore_init=False):
    folder = Path(folder)
    lines = []
    lines.append("STRUCTURE DU PROJET")
    lines.append("=" * 80)

    root_name = folder.name
    lines.append(f"[DIR]  {root_name}/")

    def list_files(dir_path, indent):
        """Liste TOUS les fichiers du dossier dir_path avec leur taille."""
        files = []
        for f in dir_path.iterdir():
            if f.is_file():
                # Appliquer ignore_init si nécessaire
                if ignore_init and f.name == '__init__.py':
                    continue
                # Récupérer la taille
                try:
                    size_bytes = f.stat().st_size
                    size_str = human_size(size_bytes)
                except Exception:
                    size_str = "?"
                # Choisir une icône selon l'extension
                ext = f.suffix.lower()
```

```
                if ext == '.py':
                    icon = "[PY] "
                elif ext == '.json':
                    icon = "[FILE] "
                elif ext == '.txt':
                    icon = "[TXT] "
                elif ext == '.po':
                    icon = "[PO] "
                elif ext == '.mo':
                    icon = "[MO] "
                elif ext in ('.html', '.htm'):
                    icon = "[HTML] "
                elif ext == '.css':
                    icon = "[CSS] "
                elif ext == '.js':
                    icon = "[JS] "
                else:
                    icon = "[FILE] "   # icône générique pour tous les autres fichiers
                files.append((f.name, size_str, icon))

        # Trier par nom
        files.sort(key=lambda x: x[0])

        for i, (name, size_str, icon) in enumerate(files):
            is_last = (i == len(files) - 1)
            prefix = "???  " if is_last else "???  "
            lines.append(f"{indent}{prefix} {icon} {name} ({size_str})")

    # Dossier racine
    list_files(folder, "")

    # Sous-dossiers (si include_subdirs est True)
    if include_subdirs:
        subdirs = sorted([d for d in folder.rglob('*') if d.is_dir()])
        for sub in subdirs:
            level = len(sub.relative_to(folder).parts)
            folder_indent = "|   " * (level - 1) if level > 1 else ""
            lines.append(f"{folder_indent}???  [DIR]  {sub.name}/")
            file_indent = "|   " * level if level > 0 else ""
            list_files(sub, file_indent)

    lines.append("\n" + "=" * 80 + "\n")
    return "\n".join(lines)

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\extractor\__init__.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\extractor\__init__.py
--------------------------------------------------------------------------------


--- FIN DU FICHIER ---
```

```
================================================================================
FICHIER Python : src\gui\extraction_controller.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\extraction_controller.py
--------------------------------------------------------------------------------
# src/gui/extraction_controller.py
# Point d'entrée vers le contrôleur principal
from src.gui.controller.main_controller import MainController as ExtractionController

__all__ = ['ExtractionController']

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\extraction_runner.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\extraction_runner.py
--------------------------------------------------------------------------------
# src/gui/extraction_runner.py
import os
from tkinter import messagebox
from pathlib import Path
from src.i18n import _
from src.logger import setup_logger
from src.services.pdf_service import PDFService  # <-- nouveau

logger = setup_logger(__name__)

def run_extraction(controller, service, selected_folder, options, selected_files,
                   progress_callback, log_callback, export_pdf=False):  # <-- nouveau paramètre
    """
    Exécute l'extraction via le service et gère les mises à jour de l'UI.
    Retourne (success, output_filename, stats) ou (False, None, None)
    Si export_pdf est True, génère également un PDF.
    """
    try:
        success, output_filename, stats = service.extract_folder(
            selected_folder,
            options,
            progress_callback=progress_callback,
            log_callback=log_callback,
            selected_files=selected_files
        )

        if not success:
            controller.ui['status_var'].set(_("Extraction échouée"))
            controller.ui['progress_var'].set(0)
            messagebox.showerror(_("Erreur"), _("Échec de l'extraction (voir journal)"))
            return False, None, None

        # Succès
        controller.ui['status_var'].set(_("Extraction terminée"))
        controller.ui['progress_var'].set(0)
```

```
        controller.add_info(_("\n--- Extraction terminée avec succès ---"))
        controller.add_info(_("Fichier créé : {}").format(output_filename))
        controller.add_info(_("Emplacement : {}").format(os.path.abspath(output_filename)))
        controller.add_info(_("Fichiers extraits : {} (Python: {}, JSON: {}, TXT: {}, PO: {}, MO: {}, HTML: {}, CSS: {
            stats['total_files'], stats['py_count'], stats['json_count'], stats['txt_count'],
            stats['po_count'], stats['mo_count'], stats['html_count'], stats['css_count'], stats['js_count']
        ))

        # --- Génération du PDF si demandé ---
        pdf_path = None
        if export_pdf:
            txt_path = Path(output_filename)
            pdf_path = txt_path.with_suffix('.pdf')
            controller.add_info(_("Génération du PDF en cours..."))
            try:
                PDFService.convert_to_pdf(txt_path, pdf_path)
                controller.add_info(_("PDF généré : {}").format(pdf_path))
                controller.add_info(_("Emplacement : {}").format(os.path.abspath(pdf_path)))
            except Exception as e:
                logger.error(f"Erreur lors de la génération du PDF : {e}")
                controller.add_info(_("Erreur de génération du PDF : {}").format(e))

        # Message de succès (inclut le PDF si généré)
        success_msg = (
            _("Extraction terminée avec succès !\n\n")
            + _("Fichier créé : {}\n").format(output_filename)
            + _("Nombre de fichiers extraits : {}\n").format(stats['total_files'])
            + _("  - Fichiers Python : {}\n").format(stats['py_count'])
            + _("  - Fichiers JSON : {}\n").format(stats['json_count'])
            + _("  - Fichiers TXT : {}\n").format(stats['txt_count'])
            + _("  - Fichiers PO : {}\n").format(stats['po_count'])
            + _("  - Fichiers MO : {}\n").format(stats['mo_count'])
            + _("  - Fichiers HTML : {}\n").format(stats['html_count'])
            + _("  - Fichiers CSS : {}\n").format(stats['css_count'])
            + _("  - Fichiers JS : {}\n").format(stats['js_count'])
            + _("Emplacement : {}").format(os.path.abspath(output_filename))
        )
        if export_pdf and pdf_path:
            success_msg += _("\n\nPDF généré : {}").format(os.path.abspath(pdf_path))
        messagebox.showinfo(_("Succès"), success_msg)
        return True, output_filename, stats

    except Exception as e:
        logger.error(f"Erreur lors de l'extraction : {e}")
        controller.ui['status_var'].set(_("Extraction échouée"))
        controller.ui['progress_var'].set(0)
        messagebox.showerror(_("Erreur"), _("Une erreur est survenue : {}").format(e))
        return False, None, None

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\file_selector.py
Type: Python
```

Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\file_selector.py

---

```
# src/gui/file_selector.py
from src.gui.selection import SelectionDialog

def select_files(parent, folder_path, options):
    """
    Ouvre la fenêtre de sélection et retourne la liste des chemins relatifs sélectionnés.
    Si l'utilisateur annule, retourne None.
    """
    dialog = SelectionDialog(parent, folder_path, options)
    parent.wait_window(dialog.window)
    return dialog.get_selected()
```

--- FIN DU FICHIER ---

==========

FICHIER Python : src\gui\folder_scanner.py

Type: Python

Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\folder_scanner.py

---

```
# src/gui/folder_scanner.py
from src.extractor.file_discovery import FileDiscoveryService
from src.config import ExtractionOptions

def scan_folder(folder, ui):
    """
    Scanne le dossier et retourne les statistiques.
    Utilise FileDiscoveryService directement pour une meilleure performance.
    """
    # Construire les options à partir de l'interface
    options = ExtractionOptions(
        include_json=ui['include_json'].get(),
        include_subdirs=ui['include_subdirs'].get(),
        include_txt=ui['include_txt'].get(),
        include_po=ui['include_po'].get(),
        include_mo=ui['include_mo'].get(),
        include_html=ui['include_html'].get(),
        include_css=ui['include_css'].get(),
        include_js=ui['include_js'].get(),
        ignore_init=ui['ignore_init'].get()
        # Les autres champs ne sont pas nécessaires pour le scan
    )

    discovery = FileDiscoveryService(options)
    files = discovery.find_files(folder)

    # Dictionnaire des types (sans 'htm' séparé)
    types = {
        'py': 0, 'json': 0, 'txt': 0, 'po': 0, 'mo': 0,
        'html': 0, 'css': 0, 'js': 0
    }

    for full_path, rel_path, ext in files:  # variables explicites, pas de _
```

```
        ext_clean = ext.lstrip('.')
        if ext_clean == 'htm':              # traiter .htm comme .html
            ext_clean = 'html'
        if ext_clean in types:
            types[ext_clean] += 1

    return {
        'total': len(files),
        'types': types
    }

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\gui.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\gui.py
--------------------------------------------------------------------------------
# src/gui/gui.py
import tkinter as tk
from src.gui.ui_builder.ui_builder import build_ui
from src.gui.extraction_controller import ExtractionController
from src.services.extraction_service import ExtractionService
from src.network.server import ReceiveServer
from src.network.discovery import DiscoveryService
from src.config import config
from src.i18n import _

class PythonCodeExtractor:
    def __init__(self, root):
        self.root = root
        self.root.title(_("Extracteur de code"))
        width = config.get('gui.window_width', 700)
        height = config.get('gui.window_height', 600)
        self.root.geometry(f"{width}x{height}")
        self.root.resizable(True, True)

        # Construire l'interface
        self.ui = build_ui(root)

        # Service et contrôleur
        self.service = ExtractionService()
        self.controller = ExtractionController(root, self.ui, service=self.service)
        self.ui['controller'] = self.controller

        # Démarrer les services réseau
        self._start_network_services()

        # Transmettre les instances réseau au contrôleur
        self.controller.set_server(self.server)
        self.controller.set_discovery(self.discovery)

        # Lier les boutons
        self._bind_buttons()
```

```
        # Gestion de la fermeture
        self.root.protocol("WM_DELETE_WINDOW", self.on_close)

    def _bind_buttons(self):
        """Lier les boutons de l'interface principale."""
        self.ui['browse_btn'].config(command=self.controller.browse_folder)
        self.ui['clear_btn'].config(command=self.controller.clear_info)
        self.ui['extract_btn'].config(command=self.controller.extract_code)
        self.ui['select_btn'].config(command=self.controller.open_selection_dialog)
        self.ui['version_btn'].config(command=self.controller.open_version_explorer)
        self.ui['network_btn'].config(command=self.controller.open_network_center)

    def _start_network_services(self):
        """Démarrer le serveur et le service de découverte réseau."""
        try:
            self.server = ReceiveServer()
            self.server.start()
            self.discovery = DiscoveryService()
            self.discovery.start_listener()
        except Exception as e:
            print(f"Erreur lors du démarrage des services réseau : {e}")

    def on_close(self):
        """Fermeture propre de l'application."""
        if hasattr(self, 'server') and self.server:
            self.server.stop()
        if hasattr(self, 'discovery') and self.discovery:
            self.discovery.stop_listener()
        self.root.destroy()

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\recent_files.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\recent_files.py
--------------------------------------------------------------------------------
# src/gui/recent_files.py
import json
import os
from pathlib import Path

RECENT_FILE = Path(__file__).parent.parent.parent / "recent_folders.json"
MAX_RECENT = 10


def load_recent_folders():
    """Charge la liste des dossiers récents depuis le fichier JSON."""
    if RECENT_FILE.exists():
        try:
            with open(RECENT_FILE, 'r', encoding='utf-8') as f:
                data = json.load(f)
                folders = data.get('folders', [])
```

```
                # Filtrer les dossiers qui n'existent plus
                return [f for f in folders if os.path.exists(f)]
        except Exception:
            return []
    return []


def save_recent_folders(folders):
    """Sauvegarde la liste des dossiers récents."""
    try:
        with open(RECENT_FILE, 'w', encoding='utf-8') as f:
            json.dump({'folders': folders[:MAX_RECENT]}, f, indent=2)
    except Exception:
        pass


def add_recent_folder(folder_path):
    """Ajoute un dossier en tête de liste et sauvegarde."""
    folders = load_recent_folders()
    # Supprimer l'entrée si déjà présente
    if folder_path in folders:
        folders.remove(folder_path)
    # Insérer en tête
    folders.insert(0, folder_path)
    # Tronquer
    folders = folders[:MAX_RECENT]
    save_recent_folders(folders)
    return folders


def remove_recent_folder(folder_path):
    """Supprime un dossier de la liste (si inexistant par exemple)."""
    folders = load_recent_folders()
    if folder_path in folders:
        folders.remove(folder_path)
        save_recent_folders(folders)
    return folders


def clear_recent_folders():
    """Supprime tous les dossiers récents."""
    save_recent_folders([])

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\__init__.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\__init__.py
--------------------------------------------------------------------------------
# src/gui/__init__.py
from .extraction_controller import ExtractionController
from .file_selector import select_files
from .extraction_runner import run_extraction
```

```
from .folder_scanner import scan_folder
from .recent_files import load_recent_folders, add_recent_folder, clear_recent_folders, remove_recent_folder
from .gui import PythonCodeExtractor
from .selection import SelectionDialog

__all__ = [
    'ExtractionController',
    'select_files',
    'run_extraction',
    'scan_folder',
    'load_recent_folders',
    'add_recent_folder',
    'clear_recent_folders',
    'remove_recent_folder',
    'PythonCodeExtractor',
    'SelectionDialog'
]

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\controller\base_controller.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\controller\base_controller.py
--------------------------------------------------------------------------------
# src/gui/controller/base_controller.py
import tkinter as tk
from src.services.extraction_service import ExtractionService
from src.logger import setup_logger
from src.i18n import _

logger = setup_logger(__name__)

class BaseController:
    """Classe de base contenant les dépendances partagées."""

    def __init__(self, root, ui_widgets, service=None):
        self.root = root
        self.ui = ui_widgets
        self.service = service or ExtractionService()
        # Utiliser un attribut privé pour éviter les conflits avec les setters
        self._selected_folder = ""
        self.selected_files = []
        self._is_extracting = False
        self.ui['controller'] = self

    @property
    def selected_folder(self):
        return self._selected_folder

    @selected_folder.setter
    def selected_folder(self, value):
        self._selected_folder = value
```

```
@property
def is_extracting(self):
    return self._is_extracting

@is_extracting.setter
def is_extracting(self, value):
    self._is_extracting = value

def add_info(self, message):
    """Ajoute un message au journal."""
    self.ui['info_text'].insert(tk.END, message + "\n")
    self.ui['info_text'].see(tk.END)
    self.root.update_idletasks()

def clear_info(self):
    """Efface le journal."""
    self.ui['info_text'].delete(1.0, tk.END)

def update_status(self, message):
    """Met à jour la barre de statut."""
    self.ui['status_var'].set(message)

def set_extracting(self, extracting):
    """Active/désactive l'état d'extraction."""
    self._is_extracting = extracting
    state = 'disabled' if extracting else 'normal'
    self.ui['extract_btn'].config(state=state)

def get_include_options(self):
    """Retourne les options d'inclusion (pour la sélection)."""
    return {
        'include_json': self.ui['include_json'].get(),
        'include_txt': self.ui['include_txt'].get(),
        'include_po': self.ui['include_po'].get(),
        'include_mo': self.ui['include_mo'].get(),
        'include_html': self.ui['include_html'].get(),
        'include_css': self.ui['include_css'].get(),
        'include_js': self.ui['include_js'].get()
    }

def get_extraction_options(self):
    """Retourne toutes les options d'extraction."""
    return {
        'include_json': self.ui['include_json'].get(),
        'include_subdirs': self.ui['include_subdirs'].get(),
        'show_file_paths': self.ui['show_file_paths'].get(),
        'include_structure': self.ui['include_structure'].get(),
        'include_txt': self.ui['include_txt'].get(),
        'include_po': self.ui['include_po'].get(),
        'include_mo': self.ui['include_mo'].get(),
        'include_html': self.ui['include_html'].get(),
        'include_css': self.ui['include_css'].get(),
        'include_js': self.ui['include_js'].get(),
        'ignore_init': self.ui['ignore_init'].get(),
```

```
            'include_statistics': self.ui['include_statistics'].get(),
            'include_file_metadata': self.ui['include_file_metadata'].get()
        }

    def reset_selection(self):
        """Réinitialise la sélection de fichiers."""
        self.selected_files = []
        self.update_status(_("Prêt"))

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\controller\extraction_controller.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\controller\extraction_controller.py
--------------------------------------------------------------------------------
# src/gui/controller/extraction_controller.py
import threading
from tkinter import messagebox
from .base_controller import BaseController
from src.gui.extraction_runner import run_extraction
from src.gui.recent_files import add_recent_folder
from src.i18n import _
from src.logger import setup_logger

logger = setup_logger(__name__)

class ExtractionController(BaseController):
    """Orchestration de l'extraction de code."""

    def open_selection_dialog(self):
        """Ouvre le dialogue de sélection des fichiers."""
        if not self._selected_folder:
            messagebox.showwarning(_("Attention"), _("Veuillez d'abord sélectionner un dossier"))
            return

        from src.gui.file_selector import select_files
        options = self.get_include_options()
        selected = select_files(self.root, self._selected_folder, options)

        if selected is not None:
            self.selected_files = selected
            count = len(selected)
            self.add_info(_("Sélection mise à jour : {} fichier(s) sélectionné(s)").format(count))
            self.update_status(_("Sélection : {} fichier(s)").format(count))
        else:
            self.add_info(_("Sélection annulée"))

    def extract_code(self, export_pdf=False):
        """Lance l'extraction du code, optionnellement avec export PDF."""
        if not self._selected_folder:
            messagebox.showwarning(_("Attention"), _("Veuillez d'abord sélectionner un dossier"))
            return
```

```
    if self.is_extracting:
        return

    self.set_extracting(True)
    self._do_extraction(export_pdf)

def _do_extraction(self, export_pdf=False):
    """Exécute l'extraction dans un thread séparé."""
    options = self.get_extraction_options()
    selected = self.selected_files if self.selected_files else None

    self.update_status(_("Extraction en cours..."))
    self.add_info(_("\n--- Extraction en cours ---") + (" (PDF)" if export_pdf else ""))

    def progress_callback(current, total):
        if total > 0:
            progress = (current / total) * 100
            self.root.after(0, lambda: self.ui['progress_var'].set(progress))

    def log_callback(msg):
        self.root.after(0, lambda: self.add_info(msg))

    def extraction_thread():
        try:
            success, output_filename, stats = run_extraction(
                controller=self,
                service=self.service,
                selected_folder=self._selected_folder,
                options=options,
                selected_files=selected,
                progress_callback=progress_callback,
                log_callback=log_callback,
                export_pdf=export_pdf
            )

            self.root.after(0, lambda: self._extraction_finished(success, output_filename, stats))
        except Exception as e:
            logger.error(f"Erreur dans le thread d'extraction : {e}")
            self.root.after(0, lambda: self._extraction_error(e))

    threading.Thread(target=extraction_thread, daemon=True).start()

def _extraction_finished(self, success, output_filename, stats):
    """Appelé après la fin de l'extraction."""
    self.ui['progress_var'].set(0)
    self.set_extracting(False)

    if success:
        self.update_status(_("Extraction terminée"))
        add_recent_folder(self._selected_folder)
        if 'update_recent_menu' in self.ui:
            self.ui['update_recent_menu']()
    else:
        self.update_status(_("Extraction échouée"))
```

```
    def _extraction_error(self, error):
        """Appelé en cas d'erreur dans le thread d'extraction."""
        self.ui['progress_var'].set(0)
        self.set_extracting(False)
        self.update_status(_("Extraction échouée"))
        messagebox.showerror(_("Erreur"), _("Une erreur est survenue : {}").format(error))

    def export_to_pdf(self):
        """Lance l'extraction et génère un PDF."""
        if not self._selected_folder:
            messagebox.showwarning(_("Attention"), _("Veuillez d'abord sélectionner un dossier"))
            return

        if self.is_extracting:
            return

        self.set_extracting(True)
        self._do_extraction(export_pdf=True)

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\controller\folder_controller.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\controller\folder_controller.py
--------------------------------------------------------------------------------
# src/gui/controller/folder_controller.py
import os
from tkinter import filedialog, messagebox
from .base_controller import BaseController
from src.gui.folder_scanner import scan_folder
from src.gui.recent_files import add_recent_folder, remove_recent_folder
from src.i18n import _

class FolderController(BaseController):
    """Gestion du dossier sélectionné et des emplacements récents."""

    def browse_folder(self):
        """Ouvre un dialogue pour sélectionner un dossier."""
        folder = filedialog.askdirectory(
            title=_("Sélectionner un dossier contenant des fichiers Python/JSON/TXT/PO/MO/HTML/CSS/JS")
        )
        if not folder:
            return

        self._set_folder(folder)
        self.reset_selection()
        self._log_folder_stats()

    def select_recent_folder(self, folder_path):
        """Sélectionne un dossier depuis les emplacements récents."""
        if not os.path.exists(folder_path):
            messagebox.showerror(_("Erreur"), _("Le dossier n'existe plus : {}").format(folder_path))
```

```
            remove_recent_folder(folder_path)
            if 'update_recent_menu' in self.ui:
                self.ui['update_recent_menu']()
            return

        self._set_folder(folder_path)
        self.reset_selection()
        self._log_folder_stats()
        self.add_info(_("Dossier récent sélectionné : {}").format(folder_path))

    def _set_folder(self, folder):
        """Définit le dossier sélectionné et met à jour l'UI."""
        self._selected_folder = folder
        self.ui['folder_path_var'].set(folder)
        self.ui['extract_btn'].config(state='normal')
        self.clear_info()

    def _log_folder_stats(self):
        """Affiche les statistiques du dossier dans le journal."""
        self.add_info(_("Dossier sélectionné : {}").format(self._selected_folder))

        stats = scan_folder(self._selected_folder, self.ui)
        self.add_info(_("Fichiers trouvés : {}").format(stats['total']))
        for ext, count in stats['types'].items():
            if count > 0:
                self.add_info(_("  - Fichiers {} : {}").format(ext.upper(), count))

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\controller\main_controller.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\controller\main_controller.py
--------------------------------------------------------------------------------
# src/gui/controller/main_controller.py
"""
Contrôleur principal qui compose tous les sous-contrôleurs.
Utilise la composition plutôt que l'héritage pour combiner les fonctionnalités.
"""
from .base_controller import BaseController
from .folder_controller import FolderController
from .extraction_controller import ExtractionController
from .server_controller import ServerController
from .navigation_controller import NavigationController


class MainController(BaseController):
    """
    Contrôleur principal qui orchestre tous les sous-contrôleurs.

    Hérite de BaseController pour les méthodes communes,
    et délègue les responsabilités spécifiques aux sous-contrôleurs.
    """
```

```
def __init__(self, root, ui_widgets, service=None):
    # CRITIQUE: Créer les sous-contrôleurs AVANT d'appeler super()
    self._folder = FolderController(root, ui_widgets, service)
    self._extraction = ExtractionController(root, ui_widgets, service)
    self._server = ServerController(root, ui_widgets, service)
    self._navigation = NavigationController(root, ui_widgets, service)

    # Appeler super() pour initialiser BaseController
    super().__init__(root, ui_widgets, service)

    # Synchroniser le serveur entre les contrôleurs
    self._sync_server_references()

def _sync_server_references(self):
    """Synchronise les références du serveur entre les sous-contrôleurs."""
    # Les sous-contrôleurs ont besoin des mêmes instances de server/discovery
    # On utilise des propriétés pour que tout reste cohérent

    # Pour extraction
    self._extraction.set_server = self._server.set_server
    self._extraction.set_discovery = self._server.set_discovery

    # Pour navigation
    self._navigation.set_server = self._server.set_server
    self._navigation.set_discovery = self._server.set_discovery

# --- Propriétés avec délégation ---

@property
def selected_folder(self):
    return self._folder.selected_folder

@selected_folder.setter
def selected_folder(self, value):
    self._folder.selected_folder = value
    self._extraction.selected_folder = value

@property
def selected_files(self):
    return self._extraction.selected_files

@selected_files.setter
def selected_files(self, value):
    self._extraction.selected_files = value

@property
def is_extracting(self):
    return self._extraction.is_extracting

@is_extracting.setter
def is_extracting(self, value):
    self._extraction.is_extracting = value

# --- Délégation FolderController ---
```

```
def browse_folder(self):
    self._folder.browse_folder()
    # Synchroniser avec extraction
    self._extraction.selected_folder = self._folder.selected_folder

def select_recent_folder(self, folder_path):
    self._folder.select_recent_folder(folder_path)
    self._extraction.selected_folder = self._folder.selected_folder

# --- Délégation ExtractionController ---
def open_selection_dialog(self):
    self._extraction.open_selection_dialog()
    # Synchroniser les fichiers sélectionnés
    self.selected_files = self._extraction.selected_files

def extract_code(self):
    self._extraction.extract_code()

def export_to_pdf(self):
    """Lance l'extraction et génère un PDF."""
    self._extraction.export_to_pdf()

def set_extracting(self, extracting):
    """Active/désactive l'état d'extraction."""
    self._extraction.set_extracting(extracting)

# --- Délégation ServerController ---
def set_server(self, server):
    self._server.set_server(server)
    self._navigation.set_server(server)

def set_discovery(self, discovery):
    self._server.set_discovery(discovery)
    self._navigation.set_discovery(discovery)

def start_server(self):
    self._server.start_server()

def stop_server(self):
    self._server.stop_server()

def get_server(self):
    return self._server.get_server()

def get_discovery(self):
    return self._server.get_discovery()

# --- Délégation NavigationController ---
def open_version_explorer(self):
    self._navigation.open_version_explorer()

def open_network_center(self):
    self._navigation.open_network_center()
```

```
    # --- Méthodes communes ---
    def add_info(self, message):
        self.ui['info_text'].insert(tk.END, message + "\n")
        self.ui['info_text'].see(tk.END)
        self.root.update_idletasks()

    def clear_info(self):
        self.ui['info_text'].delete(1.0, tk.END)

    def update_status(self, message):
        self.ui['status_var'].set(message)

    def get_include_options(self):
        return self._folder.get_include_options()

    def get_extraction_options(self):
        return self._folder.get_extraction_options()

    def reset_selection(self):
        self._folder.reset_selection()
        self._extraction.reset_selection()

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\controller\navigation_controller.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\controller\navigation_controller.py
--------------------------------------------------------------------------------
# src/gui/controller/navigation_controller.py
from tkinter import messagebox
from .base_controller import BaseController
from src.i18n import _
from src.logger import setup_logger

logger = setup_logger(__name__)

class NavigationController(BaseController):
    """Ouverture des dialogues (version explorer, réseau center)."""

    def __init__(self, root, ui_widgets, service=None):
        super().__init__(root, ui_widgets, service)
        self.server = None
        self.discovery = None

    def set_server(self, server):
        """Définit l'instance du serveur partagé."""
        self.server = server

    def set_discovery(self, discovery):
        """Définit l'instance du service de découverte partagé."""
        self.discovery = discovery

    def open_version_explorer(self):
```

```
        """Ouvre le gestionnaire de versions."""
        try:
            from src.gui.version_explorer import VersionExplorerDialog
            VersionExplorerDialog(self.root)
        except Exception as e:
            logger.error(f"Erreur lors de l'ouverture du gestionnaire de versions : {e}")
            messagebox.showerror(_("Erreur"), _("Impossible d'ouvrir le gestionnaire de versions : {}").format(e))

    def open_network_center(self):
        """Ouvre le tableau de bord réseau."""
        try:
            from src.gui.network_center import NetworkCenterDialog
            # Passer self comme contrôleur (pour que les onglets puissent appeler)
            # MAIS il faut passer le MainController qui a start_server()
            # On va chercher le contrôleur parent via self.ui['controller']
            main_controller = self.ui.get('controller')
            if main_controller:
                NetworkCenterDialog(self.root, main_controller, self.server, self.discovery)
            else:
                # Fallback: passer self (mais start_server ne sera pas disponible)
                NetworkCenterDialog(self.root, self, self.server, self.discovery)
        except Exception as e:
            logger.error(f"Erreur lors de l'ouverture du centre réseau : {e}")
            messagebox.showerror(_("Erreur"), _("Impossible d'ouvrir le centre réseau : {}").format(e))

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\controller\server_controller.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\controller\server_controller.py
--------------------------------------------------------------------------------
# src/gui/controller/server_controller.py
from .base_controller import BaseController
from src.network.server import ReceiveServer
from src.config import config
from src.logger import setup_logger

logger = setup_logger(__name__)

class ServerController(BaseController):
    """Gestion du serveur réseau (démarrage/arrêt)."""

    def __init__(self, root, ui_widgets, service=None):
        super().__init__(root, ui_widgets, service)
        self.server = None
        self.discovery = None

    def set_server(self, server):
        """Définit l'instance du serveur partagé."""
        self.server = server

    def set_discovery(self, discovery):
        """Définit l'instance du service de découverte partagé."""
```

```
        self.discovery = discovery

    def start_server(self):
        """Démarre le serveur réseau."""
        if self.server and self.server.is_alive():
            logger.info("Serveur déjà en cours")
            return

        if self.server:
            self.server.stop()

        self.server = ReceiveServer(
            host=config.get('network.server_host', '0.0.0.0'),
            port=config.get('network.server_port', 50000),
            received_dir=self.service.output_dir / "received"
        )
        self.server.start()
        logger.info("Serveur redémarré via le contrôleur")

    def stop_server(self):
        """Arrête le serveur réseau."""
        if self.server and self.server.is_alive():
            self.server.stop()
        else:
            logger.info("Serveur déjà arrêté")

    def get_server(self):
        """Retourne l'instance du serveur."""
        return self.server

    def get_discovery(self):
        """Retourne l'instance du service de découverte."""
        return self.discovery

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\controller\__init__.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\controller\__init__.py
--------------------------------------------------------------------------------
# src/gui/controller/__init__.py
from .base_controller import BaseController
from .folder_controller import FolderController
from .extraction_controller import ExtractionController
from .server_controller import ServerController
from .navigation_controller import NavigationController
from .main_controller import MainController

__all__ = [
    'BaseController',
    'FolderController',
    'ExtractionController',
    'ServerController',
```

```
    'NavigationController',
    'MainController'
]

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\network_center\dialog.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\network_center\dialog.py
--------------------------------------------------------------------------------
# src/gui/network_center/dialog.py
import tkinter as tk
from tkinter import ttk
from src.i18n import _
from src.logger import setup_logger
from .status_tab import StatusTab
from .send_tab import SendTab
from .received_tab import ReceivedTab
from .log_tab import LogTab

logger = setup_logger(__name__)

class NetworkCenterDialog(tk.Toplevel):
    def __init__(self, parent, controller, server=None, discovery=None):
        super().__init__(parent)
        self.title(_("Centre réseau"))
        self.geometry("850x650")
        self.minsize(750, 550)
        self.transient(parent)
        self.grab_set()

        self.controller = controller  # MainController
        self.server = server
        self.discovery = discovery
        self.output_dir = controller.service.output_dir if controller and hasattr(controller, 'service') else None

        self._create_widgets()
        self._subscribe_to_server()

        self.protocol("WM_DELETE_WINDOW", self._on_close)

    def _create_widgets(self):
        main = ttk.Frame(self, padding=10)
        main.pack(fill=tk.BOTH, expand=True)

        notebook = ttk.Notebook(main)
        notebook.pack(fill=tk.BOTH, expand=True)

        # Création des onglets - passer self.controller (MainController)
        self.status_tab = StatusTab(notebook, self, self.controller)
        self.send_tab = SendTab(notebook, self, self.discovery, self.output_dir)
        self.received_tab = ReceivedTab(notebook, self, self.output_dir)
        self.log_tab = LogTab(notebook, self)
```

```
        notebook.add(self.status_tab, text=_("État du serveur"))
        notebook.add(self.send_tab, text=_("Envoyer"))
        notebook.add(self.received_tab, text=_("Fichiers reçus"))
        notebook.add(self.log_tab, text=_("Journal réseau"))

        self.notebook = notebook

    def _subscribe_to_server(self):
        if self.server:
            self.server.add_observer(self._on_server_event)

    def _on_server_event(self, event_type, data):
        """Appelé dans le thread du serveur -> rediriger vers l'UI via after."""
        self.after(0, lambda: self._dispatch_event(event_type, data))

    def _dispatch_event(self, event_type, data):
        # Statut
        if hasattr(self, 'status_tab'):
            self.status_tab.on_event(event_type, data)
        # Journal
        if hasattr(self, 'log_tab'):
            self.log_tab.on_event(event_type, data)
        # Received tab peut aussi réagir (rafraîchir)
        if event_type == 'file_received' and hasattr(self, 'received_tab'):
            self.received_tab.on_file_received(data)

    def _on_close(self):
        if self.server:
            self.server.remove_observer(self._on_server_event)
        self.destroy()

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\network_center\log_tab.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\network_center\log_tab.py
--------------------------------------------------------------------------------
# src/gui/network_center/log_tab.py
import tkinter as tk
from tkinter import ttk
from datetime import datetime
from src.i18n import _

class LogTab(ttk.Frame):
    def __init__(self, parent, dialog):
        super().__init__(parent)
        self.dialog = dialog
        self.log_entries = []  # pour limiter

        self._create_widgets()

    def _create_widgets(self):
```

```
        main = ttk.Frame(self, padding=10)
        main.pack(fill=tk.BOTH, expand=True)

        self.text = tk.Text(main, wrap=tk.WORD, height=20, state=tk.DISABLED)
        self.text.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)

        scroll = ttk.Scrollbar(main, orient=tk.VERTICAL, command=self.text.yview)
        scroll.pack(side=tk.RIGHT, fill=tk.Y)
        self.text.config(yscrollcommand=scroll.set)

        btn_frame = ttk.Frame(main)
        btn_frame.pack(fill=tk.X, pady=5)

        ttk.Button(btn_frame, text=_("Effacer"), command=self._clear).pack(side=tk.LEFT, padx=2)

    def _append(self, message):
        self.text.config(state=tk.NORMAL)
        self.text.insert(tk.END, message + "\n")
        self.text.see(tk.END)
        self.text.config(state=tk.DISABLED)
        # Limiter à 500 lignes
        lines = self.text.get(1.0, tk.END).count('\n')
        if lines > 500:
            self.text.config(state=tk.NORMAL)
            self.text.delete(1.0, 2.0)  # supprime la première ligne
            self.text.config(state=tk.DISABLED)

    def _clear(self):
        self.text.config(state=tk.NORMAL)
        self.text.delete(1.0, tk.END)
        self.text.config(state=tk.DISABLED)

    def on_event(self, event_type, data):
        """Reçoit les événements du serveur et les formate."""
        timestamp = datetime.now().strftime("%H:%M:%S")
        if event_type == 'started':
            host = data.get('host', '?')
            port = data.get('port', '?')
            self._append(f"[{timestamp}] [OK]  Serveur démarré sur {host}:{port}")
        elif event_type == 'stopped':
            self._append(f"[{timestamp}] ? Serveur arrêté")
        elif event_type == 'file_received':
            filename = data.get('filename', '')
            size = data.get('size', 0)
            addr = data.get('addr', ('?', '?'))
            self._append(f"[{timestamp}] ? Fichier reçu : {filename} ({size} o) de {addr[0]}")
        elif event_type == 'rejected':
            reason = data.get('reason', 'inconnu')
            filename = data.get('filename', '')
            addr = data.get('addr', ('?', '?'))
            self._append(f"[{timestamp}] [ERR]  Rejeté : {reason} {filename} de {addr[0]}")
        elif event_type == 'start_failed':
            error = data.get('error', '')
            self._append(f"[{timestamp}] [ERR]  Échec au démarrage : {error}")
```

```
        else:
            self._append(f"[{timestamp}] ?? Événement : {event_type} {data}")

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\network_center\received_tab.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\network_center\received_tab.py
--------------------------------------------------------------------------------
# src/gui/network_center/received_tab.py
import os
import shutil
import tkinter as tk
from tkinter import ttk, messagebox
from pathlib import Path
from datetime import datetime
from src.i18n import _
from src.logger import setup_logger

logger = setup_logger(__name__)

class ReceivedTab(ttk.Frame):
    def __init__(self, parent, dialog, output_dir):
        super().__init__(parent)
        self.dialog = dialog
        self.output_dir = output_dir
        self.received_dir = output_dir / "received" if output_dir else None
        self.selected_file = None

        self._create_widgets()
        self._refresh_list()

    def _create_widgets(self):
        main = ttk.Frame(self, padding=10)
        main.pack(fill=tk.BOTH, expand=True)

        # Liste
        columns = ('name', 'size', 'date')
        self.tree = ttk.Treeview(main, columns=columns, show='headings')
        self.tree.heading('name', text=_('Nom'))
        self.tree.heading('size', text=_('Taille'))
        self.tree.heading('date', text=_('Date'))
        self.tree.column('name', width=300)
        self.tree.column('size', width=100)
        self.tree.column('date', width=150)
        self.tree.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)

        scroll = ttk.Scrollbar(main, orient=tk.VERTICAL, command=self.tree.yview)
        scroll.pack(side=tk.RIGHT, fill=tk.Y)
        self.tree.config(yscrollcommand=scroll.set)

        self.tree.bind('<<TreeviewSelect>>', self._on_select)
```

```
    # Boutons
    btn_frame = ttk.Frame(main)
    btn_frame.pack(fill=tk.X, pady=5)

    ttk.Button(btn_frame, text=_("Ouvrir"), command=self._open_file).pack(side=tk.LEFT, padx=2)
    ttk.Button(btn_frame, text=_("Ouvrir le dossier"), command=self._open_folder).pack(side=tk.LEFT, padx=2)
    ttk.Button(btn_frame, text=_("Supprimer"), command=self._delete_file).pack(side=tk.LEFT, padx=2)
    ttk.Button(btn_frame, text=_("Déplacer vers out/"), command=self._move_to_out).pack(side=tk.LEFT, padx=2)
    ttk.Button(btn_frame, text=_("Rafraîchir"), command=self._refresh_list).pack(side=tk.LEFT, padx=2)

    self.status_var = tk.StringVar(value="")
    ttk.Label(main, textvariable=self.status_var).pack(fill=tk.X, pady=5)

def _refresh_list(self):
    for item in self.tree.get_children():
        self.tree.delete(item)

    if not self.received_dir or not self.received_dir.exists():
        self.status_var.set(_("Dossier received/ introuvable"))
        return

    try:
        files = sorted(self.received_dir.iterdir(), key=lambda p: p.stat().st_mtime, reverse=True)
        for f in files:
            if f.is_file():
                size = f.stat().st_size
                size_str = self._human_size(size)
                mtime = datetime.fromtimestamp(f.stat().st_mtime).strftime("%Y-%m-%d %H:%M:%S")
                self.tree.insert('', 'end', values=(f.name, size_str, mtime), tags=(str(f),))
        self.status_var.set(_("{} fichier(s)").format(len(files)))
    except Exception as e:
        logger.error(f"Erreur lecture received : {e}")
        self.status_var.set(_("Erreur de lecture"))

def _human_size(self, size):
    for unit in ['o', 'Ko', 'Mo']:
        if size < 1024:
            return f"{size:.1f} {unit}"
        size /= 1024
    return f"{size:.1f} Go"

def _on_select(self, event):
    selection = self.tree.selection()
    if selection:
        self.selected_file = Path(self.tree.item(selection[0], 'tags')[0])
    else:
        self.selected_file = None

def _open_file(self):
    if self.selected_file and self.selected_file.exists():
        try:
            os.startfile(str(self.selected_file))
        except Exception as e:
            messagebox.showerror(_("Erreur"), _("Impossible d'ouvrir : {}").format(e))
```

```
        else:
            messagebox.showinfo(_("Information"), _("Aucun fichier sélectionné"))

    def _open_folder(self):
        if self.received_dir and self.received_dir.exists():
            try:
                os.startfile(str(self.received_dir))
            except Exception as e:
                messagebox.showerror(_("Erreur"), _("Impossible d'ouvrir le dossier : {}").format(e))

    def _delete_file(self):
        if not self.selected_file or not self.selected_file.exists():
            messagebox.showinfo(_("Information"), _("Aucun fichier sélectionné"))
            return
        if messagebox.askyesno(_("Confirmation"), _("Supprimer définitivement {} ?").format(self.selected_file.name)):
            try:
                self.selected_file.unlink()
                self._refresh_list()
                self.status_var.set(_("Fichier supprimé"))
            except Exception as e:
                messagebox.showerror(_("Erreur"), _("Suppression échouée : {}").format(e))

    def _move_to_out(self):
        if not self.selected_file or not self.selected_file.exists():
            messagebox.showinfo(_("Information"), _("Aucun fichier sélectionné"))
            return
        dest = self.output_dir / self.selected_file.name
        if dest.exists():
            # Ajouter un suffixe
            base = dest.stem
            ext = dest.suffix
            counter = 1
            while dest.exists():
                dest = self.output_dir / f"{base}_{counter}{ext}"
                counter += 1
        try:
            shutil.move(str(self.selected_file), str(dest))
            self._refresh_list()
            self.status_var.set(_("Fichier déplacé vers out/"))
        except Exception as e:
            messagebox.showerror(_("Erreur"), _("Déplacement échoué : {}").format(e))

    def on_file_received(self, data):
        # Nouveau fichier reçu -> rafraîchir la liste
        self._refresh_list()
        self.status_var.set(_("Nouveau fichier reçu : {}").format(data.get('filename', '')))

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\network_center\send_tab.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\network_center\send_tab.py
--------------------------------------------------------------------------------
```

```
# src/gui/network_center/send_tab.py
import os
import socket
import threading
import tkinter as tk
from tkinter import ttk, messagebox
from pathlib import Path
from hashlib import sha256
from src.i18n import _
from src.logger import setup_logger
from src.config import config

logger = setup_logger(__name__)

class SendTab(ttk.Frame):
    def __init__(self, parent, dialog, discovery, output_dir):
        super().__init__(parent)
        self.dialog = dialog
        self.discovery = discovery
        self.output_dir = output_dir or Path(config.get('output_dir', 'out'))
        self.selected_file = None
        self.selected_peer = None
        self.peers = {}
        self.files_map = []
        self.send_history = []  # liste de (timestamp, filename, peer, status)

        self._create_widgets()
        self._populate_files()
        self._start_scan()

    def _create_widgets(self):
        main = ttk.Frame(self, padding=10)
        main.pack(fill=tk.BOTH, expand=True)

        # Fichiers
        ttk.Label(main, text=_("Fichiers disponibles dans out/ :")).grid(row=0, column=0, sticky=tk.W)
        self.file_listbox = tk.Listbox(main, height=8, width=70)
        self.file_listbox.grid(row=1, column=0, columnspan=2, sticky=(tk.W, tk.E), pady=5)
        scroll_file = ttk.Scrollbar(main, orient=tk.VERTICAL, command=self.file_listbox.yview)
        scroll_file.grid(row=1, column=2, sticky=(tk.N, tk.S))
        self.file_listbox.config(yscrollcommand=scroll_file.set)
        self.file_listbox.bind('<<ListboxSelect>>', self._on_file_select)

        # Pairs
        ttk.Label(main, text=_("PC disponibles sur le réseau :")).grid(row=2, column=0, sticky=tk.W, pady=(10, 0))
        self.peer_listbox = tk.Listbox(main, height=6, width=70)
        self.peer_listbox.grid(row=3, column=0, columnspan=2, sticky=(tk.W, tk.E), pady=5)
        scroll_peer = ttk.Scrollbar(main, orient=tk.VERTICAL, command=self.peer_listbox.yview)
        scroll_peer.grid(row=3, column=2, sticky=(tk.N, tk.S))
        self.peer_listbox.config(yscrollcommand=scroll_peer.set)
        self.peer_listbox.bind('<<ListboxSelect>>', self._on_peer_select)

        # Boutons
        btn_frame = ttk.Frame(main)
```

```
        btn_frame.grid(row=4, column=0, columnspan=2, pady=10)

        self.send_btn = ttk.Button(btn_frame, text=_("Envoyer"), command=self._send, state='disabled')
        self.send_btn.pack(side=tk.LEFT, padx=5)

        ttk.Button(btn_frame, text=_("Rafraîchir la liste"), command=self._populate_files).pack(side=tk.LEFT, padx=5)
        ttk.Button(btn_frame, text=_("Scanner le réseau"), command=self._start_scan).pack(side=tk.LEFT, padx=5)

        # Progression
        self.progress_var = tk.DoubleVar()
        self.progress_bar = ttk.Progressbar(main, variable=self.progress_var, maximum=100, length=400)
        self.progress_bar.grid(row=5, column=0, columnspan=2, pady=5)

        # Statut
        self.status_var = tk.StringVar(value=_("Prêt"))
        ttk.Label(main, textvariable=self.status_var).grid(row=6, column=0, columnspan=2, sticky=tk.W)

        # Historique
        ttk.Label(main, text=_("Historique des envois :")).grid(row=7, column=0, sticky=tk.W, pady=(10, 0))
        self.history_text = tk.Text(main, height=4, width=70, state=tk.DISABLED, wrap=tk.WORD)
        self.history_text.grid(row=8, column=0, columnspan=2, pady=5)

        main.columnconfigure(0, weight=1)
        main.columnconfigure(1, weight=1)

    def _populate_files(self):
        self.file_listbox.delete(0, tk.END)
        self.files_map = []
        try:
            for file_path in self.output_dir.rglob('*'):
                if file_path.is_file() and file_path.suffix == '.txt':
                    display = str(file_path.relative_to(self.output_dir))
                    self.files_map.append((display, str(file_path)))
                    self.file_listbox.insert(tk.END, display)
        except Exception as e:
            logger.error(f"Erreur listage fichiers : {e}")
            self.status_var.set(_("Erreur de lecture des fichiers"))

    def _on_file_select(self, event):
        sel = self.file_listbox.curselection()
        self.selected_file = self.files_map[sel[0]][1] if sel else None
        self._update_buttons()

    def _on_peer_select(self, event):
        sel = self.peer_listbox.curselection()
        if sel:
            try:
                ip = list(self.peers.keys())[sel[0]]
                self.selected_peer = ip
            except IndexError:
                self.selected_peer = None
        else:
            self.selected_peer = None
        self._update_buttons()
```

```
def _update_buttons(self):
    self.send_btn.config(state='normal' if (self.selected_file and self.selected_peer) else 'disabled')

def _start_scan(self):
    self.status_var.set(_("Scan en cours..."))
    threading.Thread(target=self._scan_thread, daemon=True).start()

def _scan_thread(self):
    try:
        self.peers = self.discovery.scan(timeout=2) if self.discovery else {}
    except Exception as e:
        logger.error(f"Erreur scan : {e}")
        self.peers = {}
    self.dialog.after(0, self._update_peer_list)

def _update_peer_list(self):
    self.peer_listbox.delete(0, tk.END)
    for ip, hostname in self.peers.items():
        display = f"{hostname} ({ip})"
        self.peer_listbox.insert(tk.END, display)
    self.status_var.set(f"{len(self.peers)} PC trouvé(s)" if self.peers else _("Aucun PC trouvé"))

def _send(self):
    if not (self.selected_file and self.selected_peer):
        return

    self.send_btn.config(state='disabled')
    self.progress_var.set(0)
    self.status_var.set(_("Envoi en cours..."))

    threading.Thread(target=self._send_thread, daemon=True).start()

def _send_thread(self):
    ip = self.selected_peer
    filename = os.path.basename(self.selected_file)
    port = config.get('network.server_port', 50000)

    try:
        with open(self.selected_file, 'rb') as f:
            data = f.read()

        file_hash = sha256(data).digest()
        total_size = len(data)

        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        sock.settimeout(5)
        sock.connect((ip, port))

        name_bytes = filename.encode('utf-8')
        sock.sendall(len(name_bytes).to_bytes(4, 'big'))
        sock.sendall(name_bytes)
        sock.sendall(total_size.to_bytes(8, 'big'))
```

```
            # Envoi par chunks de 64 Ko
            chunk_size = 64 * 1024
            sent = 0
            while sent < total_size:
                chunk = data[sent:sent+chunk_size]
                sock.sendall(chunk)
                sent += len(chunk)
                progress = (sent / total_size) * 100
                self.dialog.after(0, lambda p=progress: self.progress_var.set(p))

            sock.sendall(file_hash)
            sock.close()

            hostname = self.peers.get(ip, ip)
            self.dialog.after(0, lambda: self._send_success(hostname, ip, filename))
        except socket.timeout:
            self.dialog.after(0, lambda: self._send_error(_("Le serveur distant ne répond pas (timeout)"), filename, i
        except ConnectionRefusedError:
            self.dialog.after(0, lambda: self._send_error(_("Le serveur distant a refusé la connexion"), filename, ip)
        except Exception as e:
            logger.error(f"Erreur envoi : {e}")
            self.dialog.after(0, lambda: self._send_error(_("Échec de l'envoi : {}").format(e), filename, ip))

    def _send_success(self, hostname, ip, filename):
        self.send_btn.config(state='normal')
        self.status_var.set(_("Envoi réussi"))
        self.progress_var.set(100)
        self._add_history(filename, f"{hostname} ({ip})", _("Succès"))
        messagebox.showinfo(_("Succès"), _("Fichier envoyé à {} ({})").format(hostname, ip))
        logger.info(f"Fichier envoyé à {hostname} ({ip})")

    def _send_error(self, error_msg, filename=None, ip=None):
        self.send_btn.config(state='normal')
        self.status_var.set(_("Échec de l'envoi"))
        self.progress_var.set(0)
        peer = f"{ip}" if ip else _("inconnu")
        self._add_history(filename or _("fichier"), peer, _("Échec"))
        messagebox.showerror(_("Erreur"), error_msg)
        logger.error(error_msg)

    def _add_history(self, filename, peer, status):
        timestamp = tk.datetime.now().strftime("%H:%M:%S") if hasattr(tk, 'datetime') else ""
        self.send_history.append((timestamp, filename, peer, status))
        if len(self.send_history) > 20:
            self.send_history.pop(0)
        self._update_history_display()

    def _update_history_display(self):
        self.history_text.config(state=tk.NORMAL)
        self.history_text.delete(1.0, tk.END)
        for ts, fname, peer, status in reversed(self.send_history):
            line = f"[{ts}] {fname} -> {peer} : {status}\n" if ts else f"{fname} -> {peer} : {status}\n"
            self.history_text.insert(tk.END, line)
        self.history_text.config(state=tk.DISABLED)
```

```
        self.history_text.see(tk.END)

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\network_center\status_tab.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\network_center\status_tab.py
--------------------------------------------------------------------------------
# src/gui/network_center/status_tab.py
import tkinter as tk
from tkinter import ttk
from src.i18n import _
from src.network.utils import get_local_ip

class StatusTab(ttk.Frame):
    def __init__(self, parent, dialog, controller):
        super().__init__(parent)
        self.dialog = dialog
        self.controller = controller  # MainController (ou celui qui a start_server)
        self.server = controller.get_server() if hasattr(controller, 'get_server') else None
        self.received_count = 0
        self.last_received = ""

        self._create_widgets()
        self._update_status()

    def _create_widgets(self):
        main = ttk.Frame(self, padding=10)
        main.pack(fill=tk.BOTH, expand=True)

        # État
        self.status_label = ttk.Label(main, text=_("État : Inconnu"))
        self.status_label.grid(row=0, column=0, sticky=tk.W, pady=5)

        self.host_label = ttk.Label(main, text=_("Hôte : -"))
        self.host_label.grid(row=1, column=0, sticky=tk.W, pady=2)

        self.port_label = ttk.Label(main, text=_("Port : -"))
        self.port_label.grid(row=2, column=0, sticky=tk.W, pady=2)

        self.ip_label = ttk.Label(main, text=_("IP locale : {}").format(get_local_ip()))
        self.ip_label.grid(row=3, column=0, sticky=tk.W, pady=2)

        # Boutons start/stop
        btn_frame = ttk.Frame(main)
        btn_frame.grid(row=4, column=0, pady=10, sticky=tk.W)

        self.start_btn = ttk.Button(btn_frame, text=_("Démarrer"), command=self._start_server)
        self.start_btn.pack(side=tk.LEFT, padx=5)

        self.stop_btn = ttk.Button(btn_frame, text=_("Arrêter"), command=self._stop_server)
        self.stop_btn.pack(side=tk.LEFT, padx=5)
```

```
        # Compteur
        ttk.Label(main, text=_("Fichiers reçus depuis le lancement :")).grid(row=5, column=0, sticky=tk.W, pady=(15, 0
        self.count_var = tk.StringVar(value="0")
        ttk.Label(main, textvariable=self.count_var).grid(row=6, column=0, sticky=tk.W)

        ttk.Label(main, text=_("Dernier reçu :")).grid(row=7, column=0, sticky=tk.W, pady=(10, 0))
        self.last_var = tk.StringVar(value="")
        ttk.Label(main, textvariable=self.last_var).grid(row=8, column=0, sticky=tk.W)

        # Separator
        ttk.Separator(main, orient='horizontal').grid(row=9, column=0, sticky=tk.EW, pady=15)

        # Info sur les fichiers reçus
        ttk.Label(main, text=_("Les fichiers reçus sont stockés dans out/received/")).grid(row=10, column=0, sticky=tk

        self._update_buttons()

    def _update_status(self):
        """Met à jour l'affichage de l'état du serveur."""
        # Vérifier si le serveur est en cours d'exécution
        server = self.controller.get_server() if hasattr(self.controller, 'get_server') else None
        is_running = server and server.is_alive()

        if is_running:
            host = getattr(server, 'host', '?')
            port = getattr(server, 'port', '?')
            self.status_label.config(text=_("État : Démarré"))
            self.host_label.config(text=_("Hôte : {}").format(host))
            self.port_label.config(text=_("Port : {}").format(port))
        else:
            self.status_label.config(text=_("État : Arrêté"))
            self.host_label.config(text=_("Hôte : -"))
            self.port_label.config(text=_("Port : -"))
        self._update_buttons()

    def _update_buttons(self):
        """Met à jour l'état des boutons."""
        server = self.controller.get_server() if hasattr(self.controller, 'get_server') else None
        is_running = server and server.is_alive()
        self.start_btn.config(state='disabled' if is_running else 'normal')
        self.stop_btn.config(state='normal' if is_running else 'disabled')

    def _start_server(self):
        """Démarre le serveur via le contrôleur."""
        if hasattr(self.controller, 'start_server'):
            self.controller.start_server()
        self._update_status()

    def _stop_server(self):
        """Arrête le serveur via le contrôleur."""
        if hasattr(self.controller, 'stop_server'):
            self.controller.stop_server()
        self._update_status()
```

```
    def on_event(self, event_type, data):
        """Reçoit les événements du serveur."""
        if event_type == 'started':
            self._update_status()
        elif event_type == 'stopped':
            self._update_status()
        elif event_type == 'file_received':
            self.received_count += 1
            self.count_var.set(str(self.received_count))
            filename = data.get('filename', '')
            if filename:
                from datetime import datetime
                timestamp = datetime.now().strftime("%H:%M:%S")
                self.last_var.set(f"{timestamp} - {filename}")

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\network_center\__init__.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\network_center\__init__.py
--------------------------------------------------------------------------------
# src/gui/network_center/__init__.py
from .dialog import NetworkCenterDialog

__all__ = ['NetworkCenterDialog']

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\selection\dialog.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\selection\dialog.py
--------------------------------------------------------------------------------
# src/gui/selection/dialog.py
import tkinter as tk
from tkinter import ttk
from src.i18n import _
from .utils import get_allowed_extensions
from .tree_builder import TreeBuilder
from .tree_controller import TreeController
from .search import SearchBar
from .toolbar import SelectionToolbar

class SelectionDialog:
    def __init__(self, parent, root_path, options):
        self.parent = parent
        self.root_path = root_path
        self.options = options
        self.allowed_extensions = get_allowed_extensions(options)

        # Fenêtre principale
        self.window = tk.Toplevel(parent)
        self.window.title(_("Sélectionner les fichiers à extraire"))
```

```
        self.window.geometry("850x600")
        self.window.minsize(700, 400)
        self.window.transient(parent)
        self.window.grab_set()

        self._create_widgets()
        self._build_tree()
        self._update_selected_count()

        # Raccourcis clavier
        self.window.bind('<Control-a>', lambda e: self.controller.select_all())
        self.window.bind('<Control-d>', lambda e: self.controller.deselect_all())
        self.window.bind('<space>', self._on_space)

    def _create_widgets(self):
        main = ttk.Frame(self.window, padding=10)
        main.pack(fill=tk.BOTH, expand=True)

        # Barre de recherche (sans tree pour l'instant)
        self.search = SearchBar(main, None, [])

        # Treeview avec 3 colonnes
        columns = ("select", "name", "size")
        self.tree = ttk.Treeview(main, columns=columns, show="tree headings")
        self.tree.heading("select", text=_("Sélection"))
        self.tree.heading("name", text=_("Nom"))
        self.tree.heading("size", text=_("Taille"))
        self.tree.column("select", width=80, anchor="center")
        self.tree.column("name", anchor="w", width=300)
        self.tree.column("size", width=100, anchor="e")

        v_scroll = ttk.Scrollbar(main, orient=tk.VERTICAL, command=self.tree.yview)
        h_scroll = ttk.Scrollbar(main, orient=tk.HORIZONTAL, command=self.tree.xview)
        self.tree.configure(yscrollcommand=v_scroll.set, xscrollcommand=h_scroll.set)

        self.tree.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
        v_scroll.pack(side=tk.RIGHT, fill=tk.Y)
        h_scroll.pack(side=tk.BOTTOM, fill=tk.X)

        # Bind des événements
        self.tree.bind("<ButtonRelease-1>", self._on_click)
        self.tree.bind("<Double-Button-1>", self._on_double_click)

        # Frame pour la toolbar (sera rempli après construction de l'arbre)
        self.toolbar_frame = ttk.Frame(main)
        self.toolbar_frame.pack(fill=tk.X, pady=5)

        # Compteur
        status_frame = ttk.Frame(main)
        status_frame.pack(fill=tk.X, pady=(5, 0))
        self.selected_count_var = tk.StringVar(value="0 fichier sélectionné")
        ttk.Label(status_frame, textvariable=self.selected_count_var).pack(side=tk.LEFT)
        ttk.Label(status_frame, text=_("(fichiers extractibles)")).pack(side=tk.LEFT, padx=(5, 0))
```

```
    # Boutons Valider / Annuler
    action_frame = ttk.Frame(main)
    action_frame.pack(fill=tk.X, pady=10)
    ttk.Button(action_frame, text=_("Valider"),
               command=self._validate).pack(side=tk.RIGHT, padx=5)
    ttk.Button(action_frame, text=_("Annuler"),
               command=self.window.destroy).pack(side=tk.RIGHT, padx=5)

    # Stocker les références (utile pour la toolbar)
    self.main = main

def _build_tree(self):
    """Construit l'arborescence via TreeBuilder."""
    builder = TreeBuilder(self.tree, self.root_path, self.options)
    data = builder.build()

    # Créer le contrôleur APRÈS avoir les données
    self.controller = TreeController(
        self.tree,
        data['file_nodes'],
        data['folder_nodes'],
        data['folder_children']
    )
    self.controller.items_state = {}

    # Mettre à jour la recherche avec les items
    self.search.tree = self.tree
    self.search.all_items = data['all_items']

    # Créer la barre d'outils dans le frame dédié
    self.toolbar = SelectionToolbar(self.toolbar_frame, self.controller, self.allowed_extensions)

def _on_click(self, event):
    region = self.tree.identify_region(event.x, event.y)
    item = self.tree.identify_row(event.y)
    if not item:
        return
    if region == "cell":
        col = self.tree.identify_column(event.x)
        if col == "#1":
            self.controller.toggle_item(item)
            self._update_selected_count()
            return
    if item in self.controller.file_nodes or item in self.controller.folder_nodes:
        self.controller.toggle_item(item)
        self._update_selected_count()

def _on_double_click(self, event):
    item = self.tree.identify_row(event.y)
    if item and item in self.controller.folder_nodes:
        current = self.tree.item(item, "open")
        self.tree.item(item, open=not current)

def _on_space(self, event):
```

```
        item = self.tree.focus()
        if item:
            self.controller.toggle_item(item)
            self._update_selected_count()
        return "break"

    def _update_selected_count(self):
        total = len(self.controller.file_nodes)
        selected = self.controller.count_selected()
        self.selected_count_var.set(f"{selected} / {total} fichier(s) sélectionné(s)")

    def _validate(self):
        self.selected_files = self.controller.get_selected_files()
        self.window.destroy()

    def get_selected(self):
        return self.selected_files if hasattr(self, 'selected_files') else []

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\selection\search.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\selection\search.py
--------------------------------------------------------------------------------
# src/gui/selection/search.py
import tkinter as tk
from tkinter import ttk
from src.i18n import _

class SearchBar:
    def __init__(self, parent, tree, all_items):
        self.tree = tree
        self.all_items = all_items
        self.var = tk.StringVar()
        # Utilisation de trace_add (moderne) au lieu de trace
        self.var.trace_add('write', self._on_search_change)

        frame = ttk.Frame(parent)
        frame.pack(fill=tk.X, pady=(0, 5))
        ttk.Label(frame, text=_("Rechercher :")).pack(side=tk.LEFT, padx=(0, 5))
        entry = ttk.Entry(frame, textvariable=self.var)
        entry.pack(side=tk.LEFT, fill=tk.X, expand=True)
        entry.focus_set()

        self.entry = entry

    def _on_search_change(self, *args):
        pattern = self.var.get().strip().lower()
        if not pattern:
            for item in self.all_items:
                self.tree.reattach(item, self.tree.parent(item), self.tree.index(item))
            return
        for item in self.all_items:
```

```
            text = self.tree.item(item, "text").lower()
            if pattern in text:
                self.tree.reattach(item, self.tree.parent(item), self.tree.index(item))
            else:
                self.tree.detach(item)

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\selection\toolbar.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\selection\toolbar.py
--------------------------------------------------------------------------------
# src/gui/selection/toolbar.py
import tkinter as tk
from tkinter import ttk
from src.i18n import _

class SelectionToolbar:
    def __init__(self, parent, controller, allowed_extensions):
        self.controller = controller
        self.allowed_extensions = allowed_extensions

        self.toolbar = ttk.Frame(parent)
        self.toolbar.pack(fill=tk.X, pady=5)

        # Boutons par extension avec icônes
        ext_buttons = [
            ('.py', "[PY] ", _("Tous les .py")),
            ('.json', "[FILE] ", _("Tous les .json")),
            ('.txt', "[TXT] ", _("Tous les .txt")),
            ('.po', "[PO] ", _("Tous les .po")),
            ('.mo', "[MO] ", _("Tous les .mo")),
            ('.html', "[HTML] ", _("Tous les .html")),
            ('.htm', "[HTML] ", _("Tous les .htm")),
            ('.css', "[CSS] ", _("Tous les .css")),
            ('.js', "[JS] ", _("Tous les .js"))
        ]

        # Stocker les boutons pour pouvoir les mettre à jour plus tard
        self.buttons = []

        for ext, icon, label in ext_buttons:
            if ext in allowed_extensions:
                btn = ttk.Button(self.toolbar, text=f"{icon} {label}")
                btn.pack(side=tk.LEFT, padx=2)
                self.buttons.append((btn, ext))

        # Boutons génériques
        self.select_all_btn = ttk.Button(self.toolbar, text=_("? Tout"))
        self.select_all_btn.pack(side=tk.LEFT, padx=2)

        self.deselect_all_btn = ttk.Button(self.toolbar, text=_("? Rien"))
        self.deselect_all_btn.pack(side=tk.LEFT, padx=2)
```

```
        self.invert_btn = ttk.Button(self.toolbar, text=_("? Inverser"))
        self.invert_btn.pack(side=tk.LEFT, padx=2)

        # Si le contrôleur est déjà défini, configurer les commandes
        if controller:
            self.set_controller(controller)

    def set_controller(self, controller):
        """Définit le contrôleur et configure les commandes des boutons."""
        self.controller = controller

        # Configurer les boutons d'extension
        for btn, ext in self.buttons:
            btn.config(command=lambda e=ext: controller.select_by_extension(e))

        # Configurer les boutons génériques
        self.select_all_btn.config(command=controller.select_all)
        self.deselect_all_btn.config(command=controller.deselect_all)
        self.invert_btn.config(command=controller.invert_selection)

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\selection\tree_builder.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\selection\tree_builder.py
--------------------------------------------------------------------------------
# src/gui/selection/tree_builder.py
from pathlib import Path
from .utils import get_allowed_extensions, format_size

class TreeBuilder:
    def __init__(self, tree, root_path, options):
        self.tree = tree
        self.root_path = Path(root_path).resolve()
        self.options = options
        self.allowed_extensions = get_allowed_extensions(options)
        self.file_nodes = {}      # rel_path -> iid
        self.folder_nodes = {}    # rel_path -> iid
        self.folder_children = {} # iid -> liste des iid enfants
        self.all_items = []       # liste de tous les iid

    def build(self):
        """Construit toute l'arborescence."""
        # Nettoyer
        for item in self.tree.get_children():
            self.tree.delete(item)

        self.file_nodes.clear()
        self.folder_nodes.clear()
        self.folder_children.clear()
        self.all_items.clear()
```

```
        # Racine
        root_name = self.root_path.name
        root_iid = self.tree.insert("", "end", text=root_name,
                                    values=("", root_name, ""), open=True)
        self.folder_nodes[str(self.root_path)] = root_iid
        self.folder_children[root_iid] = []
        self.all_items.append(root_iid)

        self._walk_directory(self.root_path, root_iid)

        # Ouvrir tous les dossiers
        for iid in self.folder_nodes.values():
            self.tree.item(iid, open=True)

        return {
            'file_nodes': self.file_nodes,
            'folder_nodes': self.folder_nodes,
            'folder_children': self.folder_children,
            'all_items': self.all_items
        }

    def _walk_directory(self, dir_path, parent_iid):
        try:
            entries = sorted(dir_path.iterdir(), key=lambda p: (not p.is_dir(), p.name))
        except PermissionError:
            return

        for path in entries:
            rel_path = path.relative_to(self.root_path)
            rel_str = str(rel_path).replace('\\', '/')

            if path.is_dir():
                iid = self.tree.insert(parent_iid, "end", text=path.name,
                                       values=("", path.name, ""), open=False)
                self.folder_nodes[rel_str] = iid
                self.folder_children.setdefault(parent_iid, []).append(iid)
                self.folder_children[iid] = []
                self.all_items.append(iid)
                self._walk_directory(path, iid)
            else:
                ext = path.suffix.lower()
                is_extractable = ext in self.allowed_extensions
                select_char = "?" if is_extractable else "?"
                size_str = format_size(path.stat().st_size) if path.exists() else "?"
                iid = self.tree.insert(parent_iid, "end", text=path.name,
                                       values=(select_char, path.name, size_str))
                self.file_nodes[rel_str] = iid
                self.folder_children.setdefault(parent_iid, []).append(iid)
                self.tree.set(iid, "select", select_char)
                self.tree.item(iid, tags=(ext,))
                self.all_items.append(iid)

--- FIN DU FICHIER ---
```

```
================================================================================
FICHIER Python : src\gui\selection\tree_controller.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\selection\tree_controller.py
--------------------------------------------------------------------------------
# src/gui/selection/tree_controller.py

class TreeController:
    def __init__(self, tree, file_nodes, folder_nodes, folder_children):
        self.tree = tree
        self.file_nodes = file_nodes          # chemin_relatif -> iid
        self.folder_nodes = folder_nodes      # chemin_relatif -> iid
        self.folder_children = folder_children  # iid_parent -> [iid_enfant]
        self.file_iids = set(file_nodes.values())      # ensemble des iid de fichiers
        self.folder_iids = set(folder_nodes.values())  # ensemble des iid de dossiers
        self.items_state = {}  # iid -> bool (True = sélectionné)

    def toggle_item(self, item):
        """Bascule l'état d'un fichier ou dossier."""
        if item in self.file_iids:
            current = self.tree.set(item, "select")
            if current == "?":   # fichier non extractible (ignoré)
                return
            new_state = not self.items_state.get(item, False)
            self.set_item_state(item, new_state)
        elif item in self.folder_iids:
            total, selected = self._count_files_in_subtree(item)
            if total > 0:
                # Si tous les fichiers sont sélectionnés, on les désélectionne,
                # sinon on les sélectionne tous.
                new_state = (selected != total)
                self.set_folder_state(item, new_state)
                self._update_parent_state(item)
        return self.items_state

    def set_item_state(self, item, state):
        """Change l'état d'un fichier."""
        if item not in self.file_iids:
            return
        self.items_state[item] = state
        self.tree.set(item, "select", "?" if state else "?")
        self._update_parent_state(item)

    def set_folder_state(self, folder_item, state):
        """Applique l'état à tous les fichiers d'un dossier (récursivement)."""
        for child in self.folder_children.get(folder_item, []):
            if child in self.file_iids:
                self.set_item_state(child, state)
            elif child in self.folder_iids:
                self.set_folder_state(child, state)
        self._update_folder_indicator(folder_item)
        self._update_parent_state(folder_item)

    def select_by_extension(self, ext):
```

```
    """Sélectionne tous les fichiers avec une extension donnée."""
    for iid in self.file_nodes.values():  # iid
        tags = self.tree.item(iid, "tags")
        if ext in tags:
            self.set_item_state(iid, True)

def select_all(self):
    """Sélectionne tous les fichiers extractibles."""
    for iid in self.file_nodes.values():
        if self.tree.set(iid, "select") != "?":
            self.set_item_state(iid, True)

def deselect_all(self):
    """Désélectionne tous les fichiers."""
    for iid in self.file_nodes.values():
        if self.tree.set(iid, "select") != "?":
            self.set_item_state(iid, False)

def invert_selection(self):
    """Inverse la sélection."""
    for iid in self.file_nodes.values():
        if self.tree.set(iid, "select") != "?":
            current = self.items_state.get(iid, False)
            self.set_item_state(iid, not current)

def get_selected_files(self):
    """Retourne la liste des chemins relatifs sélectionnés."""
    return [rel_str for rel_str, iid in self.file_nodes.items()
            if self.items_state.get(iid, False)]

def count_selected(self):
    """Retourne le nombre de fichiers sélectionnés."""
    return sum(1 for state in self.items_state.values() if state)

# --- Méthodes privées ---

def _count_files_in_subtree(self, iid):
    """
    Retourne (total_fichiers, fichiers_selectionnes) pour tout le sous?arbre.
    Parcourt récursivement les dossiers enfants.
    """
    total = 0
    selected = 0
    stack = [iid]
    while stack:
        current = stack.pop()
        if current in self.file_iids:
            total += 1
            if self.items_state.get(current, False):
                selected += 1
        elif current in self.folder_iids:
            for child in self.folder_children.get(current, []):
                stack.append(child)
    return total, selected
```

```
    def _update_parent_state(self, item):
        parent = self.tree.parent(item)
        if parent:
            self._update_folder_indicator(parent)
            self._update_parent_state(parent)

    def _update_folder_indicator(self, folder_item):
        """
        Met à jour l'indicateur (?/?/?) d'un dossier en fonction de l'état
        de tous les fichiers de son sous?arbre.
        """
        total, selected = self._count_files_in_subtree(folder_item)
        if total == 0:
            return  # pas de fichiers extractibles, on ne change rien
        if selected == 0:
            indicator = "?"
        elif selected == total:
            indicator = "?"
        else:
            indicator = "?"
        self.tree.set(folder_item, "select", indicator)

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\selection\utils.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\selection\utils.py
--------------------------------------------------------------------------------
# src/gui/selection/utils.py
from src.utils import human_size

def get_allowed_extensions(options):
    """Retourne la liste des extensions autorisées selon les options."""
    extensions = ['.py']
    if options.get('include_json', True):
        extensions.append('.json')
    if options.get('include_txt', False):
        extensions.append('.txt')
    if options.get('include_po', False):
        extensions.append('.po')
    if options.get('include_mo', False):
        extensions.append('.mo')
    if options.get('include_html', True):
        extensions.extend(['.html', '.htm'])
    if options.get('include_css', True):
        extensions.append('.css')
    if options.get('include_js', True):
        extensions.append('.js')
    return extensions

def format_size(size_bytes):
    """Retourne la taille formatée via human_size."""
```

```
    return human_size(size_bytes) if size_bytes >= 0 else "?"

def count_selected(items_state):
    """Retourne le nombre de fichiers sélectionnés."""
    return sum(1 for state in items_state.values() if state)

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\selection\__init__.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\selection\__init__.py
--------------------------------------------------------------------------------
# src/gui/selection/__init__.py
from .dialog import SelectionDialog

__all__ = ['SelectionDialog']

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\ui_builder\menus.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\ui_builder\menus.py
--------------------------------------------------------------------------------
# src/gui/ui_builder/menus.py
import tkinter as tk
from src.config import config
import json
from pathlib import Path
import sys
import subprocess
import os
from src.i18n import _
from src.gui.recent_files import load_recent_folders, clear_recent_folders

def build_menus(parent, ui):
    menubar = tk.Menu(parent)
    parent.config(menu=menubar)
    ui['menubar'] = menubar

    # --- Menu Fichier ---
    file_menu = tk.Menu(menubar, tearoff=0)
    menubar.add_cascade(label=_("Fichier"), menu=file_menu)

    # Option "Exporter en PDF"
    file_menu.add_command(
        label=_("Exporter en PDF"),
        command=lambda: _export_to_pdf(parent, ui)
    )
    file_menu.add_separator()

    file_menu.add_command(label=_("Quitter"), command=parent.destroy, accelerator="Ctrl+Q")
    parent.bind_all("<Control-q>", lambda e: parent.destroy())
```

```
# Sous-menu "Emplacements récents"
recent_menu = tk.Menu(file_menu, tearoff=0)
file_menu.add_cascade(label=_("Emplacements récents"), menu=recent_menu)
ui['recent_menu'] = recent_menu

# Fonction pour mettre à jour le sous-menu
def update_recent_menu():
    recent_menu.delete(0, tk.END)
    folders = load_recent_folders()
    if not folders:
        recent_menu.add_command(label=_("Aucun"), state='disabled')
    else:
        for f in folders:
            label = f"{os.path.basename(f)} ({f})"
            recent_menu.add_command(
                label=label,
                command=lambda path=f: _select_recent_folder(parent, ui, path)
            )
        recent_menu.add_separator()
        recent_menu.add_command(label=_("Effacer la liste"), command=_clear_recent)
ui['update_recent_menu'] = update_recent_menu

# Définition locale de _clear_recent pour avoir accès à ui
def _clear_recent():
    """Efface la liste des dossiers récents et rafraîchit le menu."""
    clear_recent_folders()
    if 'update_recent_menu' in ui:
        ui['update_recent_menu']()

# Remplir initialement
update_recent_menu()

# --- Menu Options ---
options_menu = tk.Menu(menubar, tearoff=0)
menubar.add_cascade(label=_("Options"), menu=options_menu)
options_menu.add_checkbutton(label=_("Inclure les sous-dossiers"), variable=ui['include_subdirs'])
options_menu.add_checkbutton(label=_("Afficher les chemins des fichiers"), variable=ui['show_file_paths'])
options_menu.add_separator()
options_menu.add_checkbutton(label=_("Inclure les fichiers JSON"), variable=ui['include_json'])
options_menu.add_checkbutton(label=_("Inclure les fichiers TXT"), variable=ui['include_txt'])
options_menu.add_checkbutton(label=_("Inclure les fichiers .po (traductions)"), variable=ui['include_po'])
options_menu.add_checkbutton(label=_("Inclure les fichiers .mo (compilés)"), variable=ui['include_mo'])
options_menu.add_checkbutton(label=_("Inclure les fichiers HTML"), variable=ui['include_html'])
options_menu.add_checkbutton(label=_("Inclure les fichiers CSS"), variable=ui['include_css'])
options_menu.add_checkbutton(label=_("Inclure les fichiers JavaScript"), variable=ui['include_js'])
options_menu.add_separator()
options_menu.add_checkbutton(label=_("Inclure la structure du projet"), variable=ui['include_structure'])
options_menu.add_checkbutton(label=_("Ignorer les fichiers __init__.py"), variable=ui['ignore_init'])
options_menu.add_checkbutton(label=_("Inclure les statistiques"), variable=ui['include_statistics'])
options_menu.add_checkbutton(label=_("Métadonnées détaillées par fichier"), variable=ui['include_file_metadata'])

# --- Menu Langue ---
lang_menu = tk.Menu(menubar, tearoff=0)
```

```
    menubar.add_cascade(label=_("Langue"), menu=lang_menu)
    lang_menu.add_command(label="Français", command=lambda: change_language(parent, "fr"))
    lang_menu.add_command(label="English", command=lambda: change_language(parent, "en"))

    # --- Menu Outils ---
    tools_menu = tk.Menu(menubar, tearoff=0)
    menubar.add_cascade(label=_("Outils"), menu=tools_menu)
    tools_menu.add_command(label=_("Gestionnaire de versions"), command=lambda: _open_version_explorer(parent, ui))
    ui['tools_menu'] = tools_menu
    ui['open_version_explorer_index'] = 0

    # --- Menu Vue ---
    view_menu = tk.Menu(menubar, tearoff=0)
    menubar.add_cascade(label=_("Vue"), menu=view_menu)
    view_menu.add_checkbutton(label=_("Afficher le journal"), variable=ui['log_visible'])
    ui['view_menu'] = view_menu

    # REMARQUE : Le menu "Transfert" a été supprimé car remplacé par le bouton "[PO]  Réseau..."
    # dans la barre d'outils principale.


def _export_to_pdf(parent, ui):
    """Exporte le code du projet sélectionné en PDF."""
    controller = ui.get('controller')
    if controller:
        controller.export_to_pdf()
    else:
        # Fallback
        folder = ui['folder_path_var'].get()
        if not folder:
            tk.messagebox.showwarning(_("Attention"), _("Veuillez d'abord sélectionner un dossier"))
            return
        # On tente de récupérer le contrôleur depuis la fenêtre principale
        # Dans ce cas, on utilise un message d'erreur
        tk.messagebox.showerror(_("Erreur"), _("Contrôleur non disponible pour l'export PDF"))


def _select_recent_folder(parent, ui, folder_path):
    """Sélectionne un dossier récent via le contrôleur."""
    controller = ui.get('controller')
    if controller:
        controller.select_recent_folder(folder_path)
    else:
        # Fallback
        ui['folder_path_var'].set(folder_path)
        ui['extract_btn'].config(state='normal')


def _open_version_explorer(parent, ui):
    """Ouvre le gestionnaire de versions via le contrôleur."""
    controller = ui.get('controller')
    if controller:
        controller.open_version_explorer()
    else:
```

```
        # Fallback direct
        from src.gui.version_explorer import VersionExplorerDialog
        VersionExplorerDialog(parent)


def change_language(parent, lang_code):
    """
    Change la langue de l'application.
    Écrit la nouvelle langue dans config.json, puis redémarre l'application
    proprement en utilisant sys.executable et sys.argv pour fonctionner
    quel que soit le mode d'exécution (script direct, python -m, ou exécutable).
    """
    # Déterminer le chemin du fichier config.json
    config_path = Path(__file__).parent.parent.parent / "config.json"
    if not config_path.exists():
        # Fallback si le chemin relatif ne fonctionne pas
        config_path = Path("config.json")
        if not config_path.exists():
            tk.messagebox.showerror(
                "Erreur",
                f"Fichier config.json introuvable.\nRecherché dans :\n{config_path}"
            )
            return

    try:
        with open(config_path, 'r', encoding='utf-8') as f:
            data = json.load(f)
        data['language'] = lang_code
        with open(config_path, 'w', encoding='utf-8') as f:
            json.dump(data, f, indent=2, ensure_ascii=False)

        # Redémarrer l'application avec les mêmes arguments
        parent.destroy()
        # Attendre que la fenêtre soit réellement détruite pour éviter les conflits
        subprocess.Popen([sys.executable] + sys.argv)
        sys.exit(0)
    except Exception as e:
        tk.messagebox.showerror("Erreur", f"Impossible de changer la langue : {e}")

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\ui_builder\ui_builder.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\ui_builder\ui_builder.py
--------------------------------------------------------------------------------
# src/gui/ui_builder/ui_builder.py
import tkinter as tk
from tkinter import ttk
from .menus import build_menus
from .widgets import build_widgets
from src.config import config
from src.i18n import _
```

```
def build_ui(parent):
    ui = {}

    ui['folder_path_var'] = tk.StringVar()
    ui['progress_var'] = tk.DoubleVar()
    ui['status_var'] = tk.StringVar(value=_("Prêt"))

    # Toutes les options sont initialisées depuis la configuration centralisée
    cfg = config.get_all()

    ui['include_subdirs'] = tk.BooleanVar(value=cfg.extraction.include_subdirs)
    ui['show_file_paths'] = tk.BooleanVar(value=cfg.extraction.show_file_paths)
    ui['include_json'] = tk.BooleanVar(value=cfg.extraction.include_json)
    ui['include_txt'] = tk.BooleanVar(value=cfg.extraction.include_txt)
    ui['include_po'] = tk.BooleanVar(value=cfg.extraction.include_po)
    ui['include_mo'] = tk.BooleanVar(value=cfg.extraction.include_mo)
    ui['include_html'] = tk.BooleanVar(value=cfg.extraction.include_html)
    ui['include_css'] = tk.BooleanVar(value=cfg.extraction.include_css)
    ui['include_js'] = tk.BooleanVar(value=cfg.extraction.include_js)
    ui['include_structure'] = tk.BooleanVar(value=cfg.extraction.include_structure)
    ui['ignore_init'] = tk.BooleanVar(value=cfg.extraction.ignore_init)
    ui['include_statistics'] = tk.BooleanVar(value=cfg.extraction.include_statistics)
    ui['include_file_metadata'] = tk.BooleanVar(value=cfg.extraction.include_file_metadata)

    ui['archive_old'] = tk.BooleanVar(value=False)  # option purement UI
    ui['log_visible'] = tk.BooleanVar(value=True)

    build_menus(parent, ui)
    build_widgets(parent, ui)

    return ui

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\ui_builder\widgets.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\ui_builder\widgets.py
--------------------------------------------------------------------------------
# src/gui/ui_builder/widgets.py
import tkinter as tk
from tkinter import ttk
from src.config import config
from src.i18n import _

def build_widgets(parent, ui):
    main_frame = ttk.Frame(parent, padding="10")
    main_frame.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))
    ui['main_frame'] = main_frame

    title_label = ttk.Label(main_frame, text=_("Extracteur de code Python, JSON & TXT"),
                            font=('Arial', 16, 'bold'))
    title_label.grid(row=0, column=0, columnspan=2, pady=10)
```

```
folder_label = ttk.Label(main_frame, text=_("Dossier à scanner :"))
folder_label.grid(row=1, column=0, sticky=tk.W, pady=5)

folder_entry = ttk.Entry(main_frame, textvariable=ui['folder_path_var'],
                         width=60, state='readonly')
folder_entry.grid(row=1, column=1, padx=5, pady=5, sticky=tk.W)

browse_btn = ttk.Button(main_frame, text=_("Parcourir"))
browse_btn.grid(row=1, column=2, padx=5, pady=5)
ui['browse_btn'] = browse_btn

button_frame = ttk.Frame(main_frame)
button_frame.grid(row=2, column=0, columnspan=3, pady=10)

extract_btn = ttk.Button(button_frame, text=_("Extraire le code"), state='disabled')
extract_btn.pack(side=tk.LEFT, padx=5)
ui['extract_btn'] = extract_btn

# Bouton "Sélectionner..."
select_btn = ttk.Button(button_frame, text=_("Sélectionner..."))
select_btn.pack(side=tk.LEFT, padx=5)
ui['select_btn'] = select_btn

# Bouton "Gérer les versions"
version_btn = ttk.Button(button_frame, text=_("? Gérer les versions"))
version_btn.pack(side=tk.LEFT, padx=5)
ui['version_btn'] = version_btn

# NOUVEAU : Bouton "[PO]  Réseau..."
network_btn = ttk.Button(button_frame, text=_("[PO]  Réseau..."))
network_btn.pack(side=tk.LEFT, padx=5)
ui['network_btn'] = network_btn

clear_btn = ttk.Button(button_frame, text=_("Effacer le journal"))
clear_btn.pack(side=tk.LEFT, padx=5)
ui['clear_btn'] = clear_btn

progress_bar = ttk.Progressbar(main_frame, variable=ui['progress_var'],
                               maximum=100, length=500)
progress_bar.grid(row=3, column=0, columnspan=3, pady=10)
ui['progress_bar'] = progress_bar

info_frame = ttk.LabelFrame(main_frame, text=_("Journal"), padding="5")
info_frame.grid(row=4, column=0, columnspan=3, sticky=(tk.W, tk.E, tk.N, tk.S), pady=5)
ui['info_frame'] = info_frame

log_height = config.get('gui.log_height', 12)
info_text = tk.Text(info_frame, height=log_height, width=80, wrap=tk.WORD)
info_text.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))

scrollbar = ttk.Scrollbar(info_frame, orient="vertical", command=info_text.yview)
scrollbar.grid(row=0, column=1, sticky=(tk.N, tk.S))
info_text.configure(yscrollcommand=scrollbar.set)
ui['info_text'] = info_text
```

```
    status_bar = ttk.Label(main_frame, textvariable=ui['status_var'],
                           relief=tk.SUNKEN, anchor=tk.W)
    status_bar.grid(row=5, column=0, columnspan=3, sticky=(tk.W, tk.E), pady=5)
    ui['status_bar'] = status_bar

    main_frame.columnconfigure(1, weight=1)
    main_frame.rowconfigure(4, weight=1)
    info_frame.columnconfigure(0, weight=1)
    info_frame.rowconfigure(0, weight=1)
    parent.columnconfigure(0, weight=1)
    parent.rowconfigure(0, weight=1)

    def toggle_log_visibility():
        if ui['log_visible'].get():
            ui['info_frame'].grid()
        else:
            ui['info_frame'].grid_remove()

    view_menu = ui.get('view_menu')
    if view_menu:
        view_menu.entryconfig(0, command=toggle_log_visibility)
```

--- FIN DU FICHIER ---

==
FICHIER Python : src\gui\ui_builder__init__.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\ui_builder__init__.py
----

--- FIN DU FICHIER ---

==
FICHIER Python : src\gui\version_explorer\dialog.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\version_explorer\dialog.py
----

```
# src/gui/version_explorer/dialog.py
import os
import threading
import tkinter as tk
from tkinter import ttk, messagebox
from pathlib import Path

from src.i18n import _
from src.logger import setup_logger
from src.services.version_service import VersionArchiveService
from .project_panel import ProjectPanel
from .version_tree import VersionTree
from .preview_pane import PreviewPane
from .toolbar import Toolbar
```

```
logger = setup_logger(__name__)


class VersionExplorerDialog(tk.Toplevel):
    """Fenêtre de gestion des versions d'export."""

    def __init__(self, parent, service=None):
        super().__init__(parent)
        self.title(_("Gestionnaire de versions"))
        self.geometry("1000x650")
        self.minsize(800, 500)
        self.transient(parent)
        self.grab_set()

        self.service = service or VersionArchiveService()
        self.projects_data = {}  # {projet: [VersionEntry, ...]}
        self.current_project = None
        self.preview_entry = None

        self._create_widgets()
        # Lancer le scan dans un thread
        self._start_scan()

        # Bind pour la fermeture
        self.protocol("WM_DELETE_WINDOW", self._on_close)

    def _create_widgets(self):
        main = ttk.Frame(self, padding=10)
        main.pack(fill=tk.BOTH, expand=True)

        # Panneau gauche : projets
        self.project_panel = ProjectPanel(main, self._on_project_selected)

        # Panneau droit : split vertical haut (table) et bas (aperçu)
        right_pane = ttk.Frame(main)
        right_pane.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)

        # Table des versions
        self.version_tree = VersionTree(right_pane, self._on_versions_selected, self._on_version_double_click)

        # Aperçu
        self.preview_pane = PreviewPane(right_pane)

        # Barre d'outils (sera créée après pour avoir accès au contrôleur)
        # On va utiliser un contrôleur interne pour les actions
        self.toolbar = Toolbar(right_pane, self)  # self comme controller

        # Statut en bas
        self.status_var = tk.StringVar(value=_("Scan en cours..."))
        status_label = ttk.Label(main, textvariable=self.status_var, relief=tk.SUNKEN, anchor=tk.W)
        status_label.pack(side=tk.BOTTOM, fill=tk.X, pady=(5, 0))

    def _start_scan(self):
        """Lance le scan des projets dans un thread."""
```

```
    self.status_var.set(_("Scan des fichiers en cours..."))
    threading.Thread(target=self._scan_thread, daemon=True).start()

def _scan_thread(self):
    try:
        self.projects_data = self.service.scan_projects()
    except Exception as e:
        logger.error(f"Erreur lors du scan : {e}")
        self.after(0, lambda: self._scan_error(str(e)))
        return
    self.after(0, self._scan_finished)

def _scan_finished(self):
    self.status_var.set(_("Scan terminé"))
    self.project_panel.update_projects(self.projects_data)
    # Sélectionner le premier projet s'il y en a
    if self.projects_data:
        first = list(self.projects_data.keys())[0]
        self._on_project_selected(first)

def _scan_error(self, error_msg):
    self.status_var.set(_("Erreur de scan : {}").format(error_msg))
    messagebox.showerror(_("Erreur"), _("Impossible de scanner les fichiers : {}").format(error_msg))

def _on_project_selected(self, project_name):
    self.current_project = project_name
    entries = self.projects_data.get(project_name, [])
    self.version_tree.populate(entries)
    # Effacer l'aperçu
    self.preview_pane.show_entry(None)
    self.status_var.set(_("Projet : {} ({} versions)").format(project_name, len(entries)))

def _on_versions_selected(self, selected_entries):
    # Mise à jour de l'aperçu : afficher la première sélectionnée
    if selected_entries:
        self.preview_entry = selected_entries[0]
        self.preview_pane.show_entry(selected_entries[0])
    else:
        self.preview_entry = None
        self.preview_pane.show_entry(None)

def _on_version_double_click(self, entry):
    """Ouvre le fichier avec l'application par défaut."""
    try:
        os.startfile(str(entry.path))
    except Exception as e:
        logger.error(f"Impossible d'ouvrir {entry.path} : {e}")
        messagebox.showerror(_("Erreur"), _("Impossible d'ouvrir le fichier : {}").format(e))

# --- Actions ---

def archive_selected(self):
    entries = self.version_tree.get_selected_entries()
    if not entries:
```

```
            messagebox.showinfo(_("Information"), _("Aucune version sélectionnée."))
            return
        # Filtrer les actives
        to_archive = [e for e in entries if e.status == 'active']
        if not to_archive:
            messagebox.showinfo(_("Information"), _("Les versions sélectionnées sont déjà archivées."))
            return
        if not messagebox.askyesno(_("Confirmation"),
                                   _("Archiver {} version(s) ?").format(len(to_archive))):
            return

        # Archiver dans un thread
        self.status_var.set(_("Archivage en cours..."))
        threading.Thread(target=self._archive_thread, args=(to_archive,), daemon=True).start()

    def _archive_thread(self, entries):
        try:
            for entry in entries:
                self.service.archive(entry)
            self.after(0, self._refresh_after_action)
        except Exception as e:
            logger.error(f"Erreur lors de l'archivage : {e}")
            self.after(0, lambda: self._show_error(_("Erreur d'archivage"), str(e)))

    def restore_selected(self):
        entries = self.version_tree.get_selected_entries()
        if not entries:
            messagebox.showinfo(_("Information"), _("Aucune version sélectionnée."))
            return
        to_restore = [e for e in entries if e.status == 'archived']
        if not to_restore:
            messagebox.showinfo(_("Information"), _("Les versions sélectionnées sont déjà actives."))
            return
        if not messagebox.askyesno(_("Confirmation"),
                                   _("Restaurer {} version(s) ?").format(len(to_restore))):
            return

        self.status_var.set(_("Restauration en cours..."))
        threading.Thread(target=self._restore_thread, args=(to_restore,), daemon=True).start()

    def _restore_thread(self, entries):
        try:
            for entry in entries:
                self.service.restore(entry)
            self.after(0, self._refresh_after_action)
        except Exception as e:
            logger.error(f"Erreur lors de la restauration : {e}")
            self.after(0, lambda: self._show_error(_("Erreur de restauration"), str(e)))

    def delete_selected(self):
        entries = self.version_tree.get_selected_entries()
        if not entries:
            messagebox.showinfo(_("Information"), _("Aucune version sélectionnée."))
            return
```

```
    if not messagebox.askyesno(_("Confirmation"),
                               _("Supprimer définitivement {} version(s) ? Cette action est irréversible.").format
        return

    self.status_var.set(_("Suppression en cours..."))
    threading.Thread(target=self._delete_thread, args=(entries,), daemon=True).start()

def _delete_thread(self, entries):
    try:
        for entry in entries:
            self.service.delete(entry)
        self.after(0, self._refresh_after_action)
    except Exception as e:
        logger.error(f"Erreur lors de la suppression : {e}")
        self.after(0, lambda: self._show_error(_("Erreur de suppression"), str(e)))

def reset_counter(self):
    if not self.current_project:
        messagebox.showinfo(_("Information"), _("Aucun projet sélectionné."))
        return
    if not messagebox.askyesno(_("Confirmation"),
                               _("Réinitialiser le compteur de versions pour le projet '{}' ?").format(self.curren
        return
    try:
        self.service.reset_project(self.current_project)
        self.status_var.set(_("Compteur réinitialisé pour {}").format(self.current_project))
        # Recharger le projet
        self._refresh_after_action()
    except Exception as e:
        logger.error(f"Erreur lors de la réinitialisation : {e}")
        self._show_error(_("Erreur"), str(e))

def open_folder(self):
    if not self.current_project:
        messagebox.showinfo(_("Information"), _("Aucun projet sélectionné."))
        return
    # Ouvrir le dossier out/ (ou le dossier du projet)
    folder = self.service.output_dir
    if not folder.exists():
        messagebox.showerror(_("Erreur"), _("Le dossier de sortie n'existe pas."))
        return
    try:
        os.startfile(str(folder))
    except Exception as e:
        logger.error(f"Impossible d'ouvrir {folder} : {e}")
        messagebox.showerror(_("Erreur"), _("Impossible d'ouvrir le dossier : {}").format(e))

def refresh(self):
    """Rafraîchit la liste des projets."""
    self.status_var.set(_("Rafraîchissement..."))
    threading.Thread(target=self._refresh_thread, daemon=True).start()

def _refresh_thread(self):
    try:
```

```
            self.projects_data = self.service.scan_projects()
        except Exception as e:
            logger.error(f"Erreur lors du rafraîchissement : {e}")
            self.after(0, lambda: self._show_error(_("Erreur de rafraîchissement"), str(e)))
            return
        self.after(0, self._refresh_after_action)

    def _refresh_after_action(self):
        """Rafraîchit l'interface après une action (archivage, etc)."""
        # Re-scanner pour mettre à jour les données
        try:
            self.projects_data = self.service.scan_projects()
        except Exception as e:
            logger.error(f"Erreur lors du re-scan : {e}")
            self._show_error(_("Erreur"), str(e))
            return
        self.project_panel.update_projects(self.projects_data)
        # Resélectionner le projet courant si toujours présent
        if self.current_project in self.projects_data:
            self._on_project_selected(self.current_project)
        else:
            # Si le projet a disparu (ex: toutes les versions supprimées), sélectionner le premier
            if self.projects_data:
                first = list(self.projects_data.keys())[0]
                self._on_project_selected(first)
            else:
                self.version_tree.clear()
                self.preview_pane.show_entry(None)
        self.status_var.set(_("Mise à jour terminée"))

    def select_all(self):
        self.version_tree.select_all()

    def deselect_all(self):
        self.version_tree.deselect_all()

    def _show_error(self, title, msg):
        messagebox.showerror(title, msg)

    def _on_close(self):
        self.destroy()

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\version_explorer\preview_pane.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\version_explorer\preview_pane.py
--------------------------------------------------------------------------------
# src/gui/version_explorer/preview_pane.py
import tkinter as tk
from tkinter import ttk
from src.i18n import _
from .utils import parse_date_from_header, get_file_stats
```

```
class PreviewPane:
    """Panneau d'aperçu (en-tête + statistiques)."""

    def __init__(self, parent):
        frame = ttk.LabelFrame(parent, text=_("Aperçu"), padding=5)
        frame.pack(fill=tk.BOTH, expand=True, padx=(5, 0))

        self.text = tk.Text(frame, wrap=tk.WORD, height=10, width=40)
        self.text.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)

        scroll = ttk.Scrollbar(frame, orient=tk.VERTICAL, command=self.text.yview)
        scroll.pack(side=tk.RIGHT, fill=tk.Y)
        self.text.config(yscrollcommand=scroll.set)
        self.text.config(state=tk.DISABLED)

    def show_entry(self, entry):
        """Affiche l'aperçu de l'entrée."""
        self.text.config(state=tk.NORMAL)
        self.text.delete(1.0, tk.END)
        if not entry:
            self.text.insert(tk.END, _("Aucune version sélectionnée"))
            self.text.config(state=tk.DISABLED)
            return

        # Lire le fichier et extraire l'en-tête et le bloc STATISTIQUES
        try:
            with open(entry.path, 'r', encoding='utf-8') as f:
                content = f.read()
        except Exception as e:
            self.text.insert(tk.END, f"Erreur de lecture : {e}")
            self.text.config(state=tk.DISABLED)
            return

        # Extraire l'en-tête (jusqu'à la première ligne "--- FIN DE LA STRUCTURE ---" ou "--- FIN DES FICHIERS ---")
        lines = content.splitlines()
        preview_lines = []
        in_stats = False
        for line in lines:
            if line.startswith('--- FIN DE LA STRUCTURE ---') or line.startswith('--- FIN DES FICHIERS ---'):
                # On s'arrête après avoir ajouté les lignes précédentes
                break
            if line.startswith('STATISTIQUES'):
                in_stats = True
            preview_lines.append(line)
            # On garde aussi quelques lignes après les stats
            if in_stats and line.startswith('Volume total extrait'):
                # On peut ajouter encore quelques lignes mais on s'arrête là
                break

        preview = '\n'.join(preview_lines[:50])  # limiter à 50 lignes
        if len(preview_lines) > 50:
            preview += '\n... (tronqué)'
```

```
        self.text.insert(tk.END, preview)
        self.text.config(state=tk.DISABLED)

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\version_explorer\project_panel.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\version_explorer\project_panel.py
--------------------------------------------------------------------------------
# src/gui/version_explorer/project_panel.py
import tkinter as tk
from tkinter import ttk
from src.i18n import _


class ProjectPanel:
    """Panneau gauche affichant la liste des projets avec compteurs."""

    def __init__(self, parent, on_project_select):
        self.on_project_select = on_project_select
        self.projects = {}  # nom_projet -> compteur

        frame = ttk.LabelFrame(parent, text=_("Projets"), padding=5)
        frame.pack(side=tk.LEFT, fill=tk.Y, padx=(0, 5))

        # Listbox avec scrollbar
        self.listbox = tk.Listbox(frame, height=20, width=25)
        self.listbox.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)

        scrollbar = ttk.Scrollbar(frame, orient=tk.VERTICAL, command=self.listbox.yview)
        scrollbar.pack(side=tk.RIGHT, fill=tk.Y)
        self.listbox.config(yscrollcommand=scrollbar.set)

        self.listbox.bind('<<ListboxSelect>>', self._on_select)

    def update_projects(self, projects: dict):
        """Met à jour la liste des projets."""
        self.projects = projects
        self.listbox.delete(0, tk.END)
        for project, entries in projects.items():
            count = len(entries)
            display = f"{project} ({count})"
            self.listbox.insert(tk.END, display)

    def _on_select(self, event):
        selection = self.listbox.curselection()
        if selection:
            index = selection[0]
            project_name = list(self.projects.keys())[index]
            self.on_project_select(project_name)

    def get_selected_project(self):
        """Retourne le nom du projet sélectionné, ou None."""
```

```
        sel = self.listbox.curselection()
        if sel:
            idx = sel[0]
            return list(self.projects.keys())[idx]
        return None

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\version_explorer\toolbar.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\version_explorer\toolbar.py
--------------------------------------------------------------------------------
# src/gui/version_explorer/toolbar.py
import tkinter as tk
from tkinter import ttk
from src.i18n import _


class Toolbar:
    """Barre d'outils avec les actions principales."""

    def __init__(self, parent, controller):
        self.controller = controller
        frame = ttk.Frame(parent)
        frame.pack(fill=tk.X, pady=5)

        self.archive_btn = ttk.Button(frame, text=_("Archiver"), command=self.controller.archive_selected)
        self.archive_btn.pack(side=tk.LEFT, padx=2)

        self.restore_btn = ttk.Button(frame, text=_("Restaurer"), command=self.controller.restore_selected)
        self.restore_btn.pack(side=tk.LEFT, padx=2)

        self.delete_btn = ttk.Button(frame, text=_("Supprimer"), command=self.controller.delete_selected)
        self.delete_btn.pack(side=tk.LEFT, padx=2)

        self.open_folder_btn = ttk.Button(frame, text=_("Ouvrir dossier"), command=self.controller.open_folder)
        self.open_folder_btn.pack(side=tk.LEFT, padx=2)

        self.refresh_btn = ttk.Button(frame, text=_("Rafraîchir"), command=self.controller.refresh)
        self.refresh_btn.pack(side=tk.LEFT, padx=2)

        self.reset_btn = ttk.Button(frame, text=_("Réinitialiser compteur"), command=self.controller.reset_counter)
        self.reset_btn.pack(side=tk.LEFT, padx=2)

        # Boutons de sélection (ajoutés après)
        self.select_all_btn = ttk.Button(frame, text=_("Tout sélectionner"), command=self.controller.select_all)
        self.select_all_btn.pack(side=tk.LEFT, padx=2)

        self.deselect_all_btn = ttk.Button(frame, text=_("Tout désélectionner"), command=self.controller.deselect_all)
        self.deselect_all_btn.pack(side=tk.LEFT, padx=2)

--- FIN DU FICHIER ---
```

```
================================================================================
FICHIER Python : src\gui\version_explorer\utils.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\version_explorer\utils.py
--------------------------------------------------------------------------------
# src/gui/version_explorer/utils.py
import re
from pathlib import Path
from src.utils import human_size


def parse_date_from_header(file_path: Path) -> str:
    """Extrait la date d'extraction depuis l'en-tête du fichier."""
    try:
        with open(file_path, 'r', encoding='utf-8') as f:
            for line in f:
                if line.startswith('Date d\'extraction'):
                    parts = line.split(':', 1)
                    if len(parts) == 2:
                        return parts[1].strip()
    except Exception:
        pass
    return ""


def format_size(size_bytes: int) -> str:
    """Utilise la fonction human_size existante."""
    return human_size(size_bytes)


def get_file_stats(file_path: Path) -> dict:
    """Extrait les statistiques depuis le bloc STATISTIQUES."""
    stats = {'file_count': 0, 'line_count': 0, 'total_size': 0}
    try:
        with open(file_path, 'r', encoding='utf-8') as f:
            content = f.read()
        # Recherche du bloc STATISTIQUES (après le marqueur)
        match = re.search(r'STATISTIQUES\n=+.*?\n(.*?)(?=\n--- FIN DES FICHIERS|---|\Z)', content, re.DOTALL)
        if match:
            block = match.group(1)
            # Extraire les valeurs
            fc = re.search(r'Nombre total de fichiers\s*:\s*(\d+)', block)
            if fc:
                stats['file_count'] = int(fc.group(1))
            lc = re.search(r'Nombre total de lignes\s*:\s*(\d+)', block)
            if lc:
                stats['line_count'] = int(lc.group(1))
            # Taille : "Volume total extrait : 123.45 Ko" ou "... Mo"
            size_match = re.search(r'Volume total extrait\s*:\s*([\d.]+)\s*([KMG]o)?', block)
            if size_match:
                val = float(size_match.group(1))
                unit = size_match.group(2) or 'octets'
                if unit.startswith('K'):
                    stats['total_size'] = int(val * 1024)
```

```
                elif unit.startswith('M'):
                    stats['total_size'] = int(val * 1024 * 1024)
                else:
                    stats['total_size'] = int(val)
    except Exception:
        pass
    return stats

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\version_explorer\version_tree.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\version_explorer\version_tree.py
--------------------------------------------------------------------------------
# src/gui/version_explorer/version_tree.py
import tkinter as tk
from tkinter import ttk
from src.i18n import _
from src.utils import human_size
from .utils import parse_date_from_header, get_file_stats


class VersionTree:
    """Table des versions avec colonnes et sélection multiple."""

    def __init__(self, parent, on_version_select, on_version_double_click):
        self.on_version_select = on_version_select
        self.on_version_double_click = on_version_double_click
        self.current_entries = []
        self.selected_items = set()  # pour suivre les IID sélectionnés
        self._entry_map = {}  # iid -> VersionEntry

        self.tree = ttk.Treeview(
            parent,
            columns=('version', 'date', 'size', 'files', 'lines', 'status'),
            show='tree headings',
            selectmode='extended'  # sélection multiple
        )
        self.tree.heading('#0', text=_('Sélection'))
        self.tree.heading('version', text=_('Version'))
        self.tree.heading('date', text=_('Date'))
        self.tree.heading('size', text=_('Taille'))
        self.tree.heading('files', text=_('Fichiers'))
        self.tree.heading('lines', text=_('Lignes'))
        self.tree.heading('status', text=_('Statut'))

        self.tree.column('#0', width=80, anchor='center')
        self.tree.column('version', width=80, anchor='center')
        self.tree.column('date', width=150, anchor='w')
        self.tree.column('size', width=100, anchor='e')
        self.tree.column('files', width=80, anchor='center')
        self.tree.column('lines', width=80, anchor='center')
        self.tree.column('status', width=100, anchor='w')
```

```
    # Bind events
    self.tree.bind('<<TreeviewSelect>>', self._on_select)
    self.tree.bind('<Double-1>', self._on_double_click)

    # Scrollbar
    scroll = ttk.Scrollbar(parent, orient=tk.VERTICAL, command=self.tree.yview)
    self.tree.configure(yscrollcommand=scroll.set)

    # Pack
    self.tree.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
    scroll.pack(side=tk.RIGHT, fill=tk.Y)

def populate(self, entries):
    """Remplit le treeview avec les entrées d'un projet."""
    self.clear()
    self.current_entries = sorted(entries, key=lambda e: e.version, reverse=True)
    for entry in self.current_entries:
        # Sélection par défaut (non sélectionné)
        select_char = '?'
        item = self.tree.insert(
            '',
            'end',
            text=select_char,  # La colonne #0 est définie par 'text'
            values=(
                f"v{entry.version}",
                entry.date,
                human_size(entry.size),
                entry.file_count,
                entry.line_count,
                _(entry.status.capitalize())
            )
        )
        # Stocker l'entry associée à l'item
        self._entry_map[item] = entry

def clear(self):
    for item in self.tree.get_children():
        self.tree.delete(item)
    self.current_entries = []
    self.selected_items.clear()
    self._entry_map.clear()

def _on_select(self, event):
    selected_iids = self.tree.selection()
    # Mettre à jour les glyphes : on garde les sélectionnés avec ?
    for item in self.tree.get_children():
        if item in selected_iids:
            self.tree.item(item, text='?')
            self.selected_items.add(item)
        else:
            self.tree.item(item, text='?')
            self.selected_items.discard(item)
    # Appeler le callback avec la liste des entrées sélectionnées
```

```
        selected_entries = []
        for iid in selected_iids:
            entry = self._entry_map.get(iid)
            if entry:
                selected_entries.append(entry)
        self.on_version_select(selected_entries)

    def _on_double_click(self, event):
        item = self.tree.identify_row(event.y)
        if item:
            entry = self._entry_map.get(item)
            if entry:
                self.on_version_double_click(entry)

    def get_selected_entries(self):
        """Retourne la liste des VersionEntry sélectionnés (cochés)."""
        selected = []
        for iid in self.selected_items:
            entry = self._entry_map.get(iid)
            if entry:
                selected.append(entry)
        return selected

    def get_all_entries(self):
        return self.current_entries

    def select_all(self):
        for item in self.tree.get_children():
            self.tree.selection_add(item)
            self.tree.item(item, text='?')
            self.selected_items.add(item)
        # Mettre à jour le callback avec toutes les entrées
        all_entries = [self._entry_map.get(item) for item in self.tree.get_children() if item in self._entry_map]
        self.on_version_select(all_entries)

    def deselect_all(self):
        for item in self.tree.get_children():
            self.tree.selection_remove(item)
            self.tree.item(item, text='?')
            self.selected_items.discard(item)
        self.on_version_select([])

    def refresh(self):
        # Repeupler avec les mêmes entrées (les métadonnées peuvent avoir changé)
        entries = self.current_entries
        self.populate(entries)

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\gui\version_explorer\__init__.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\gui\version_explorer\__init__.py
--------------------------------------------------------------------------------
```

```
# src/gui/version_explorer/__init__.py
from .dialog import VersionExplorerDialog

__all__ = ['VersionExplorerDialog']

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\network\discovery.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\network\discovery.py
--------------------------------------------------------------------------------
# src/network/discovery.py
import socket
import threading
import time
from src.config import config

class DiscoveryService:
    """Découvre les autres instances du programme sur le réseau local."""

    def __init__(self, listen_port=None):
        self.listen_port = listen_port or config.get('network.discovery_port', 50001)
        self._peers = {}
        self._lock = threading.Lock()
        self.broadcast_msg = config.get('network.broadcast_msg', 'PYEXTRACTOR_DISCOVER').encode()
        self.reply_msg = config.get('network.reply_msg', 'PYEXTRACTOR_HERE').encode()
        self._stop_event = threading.Event()
        self._listener_thread = None

    def start_listener(self):
        """Démarre le thread d'écoute des messages de découverte."""
        if self._listener_thread is None or not self._listener_thread.is_alive():
            self._stop_event.clear()
            self._listener_thread = threading.Thread(target=self._listen, daemon=True)
            self._listener_thread.start()

    def stop_listener(self):
        """Arrête proprement le thread d'écoute."""
        self._stop_event.set()
        if self._listener_thread and self._listener_thread.is_alive():
            self._listener_thread.join(timeout=1.0)

    def _listen(self):
        with socket.socket(socket.AF_INET, socket.SOCK_DGRAM, socket.IPPROTO_UDP) as sock:
            sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
            sock.bind(('', self.listen_port))
            sock.settimeout(0.5)  # pour vérifier régulièrement _stop_event
            while not self._stop_event.is_set():
                try:
                    data, addr = sock.recvfrom(1024)
                except socket.timeout:
                    continue
                except Exception:
```

```
                    break
                if data == self.broadcast_msg:
                    hostname = socket.gethostname()
                    reply = f"{self.reply_msg.decode()}:{hostname}".encode()
                    sock.sendto(reply, addr)
                elif data.startswith(self.reply_msg):
                    parts = data.decode().split(':', 1)
                    if len(parts) == 2:
                        hostname = parts[1]
                        with self._lock:
                            self._peers[addr[0]] = hostname

    def scan(self, timeout=2):
        """
        Envoie un message broadcast et attend les réponses des autres instances.
        Retourne un dictionnaire {ip: hostname}.
        """
        self._peers.clear()
        with socket.socket(socket.AF_INET, socket.SOCK_DGRAM, socket.IPPROTO_UDP) as sock:
            sock.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1)
            sock.settimeout(timeout)
            sock.bind(('', 0))
            sock.sendto(self.broadcast_msg, ('<broadcast>', self.listen_port))
            start = time.time()
            while time.time() - start < timeout:
                try:
                    data, addr = sock.recvfrom(1024)
                    if data.startswith(self.reply_msg):
                        parts = data.decode().split(':', 1)
                        if len(parts) == 2:
                            hostname = parts[1]
                            with self._lock:
                                self._peers[addr[0]] = hostname
                except socket.timeout:
                    break
        with self._lock:
            return dict(self._peers)

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\network\server.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\network\server.py
--------------------------------------------------------------------------------
# src/network/server.py (modifications)
import socket
import threading
import re
from pathlib import Path
from datetime import datetime
from concurrent.futures import ThreadPoolExecutor
from hashlib import sha256
from src.logger import setup_logger
```

```
from src.config import config

logger = setup_logger(__name__)

class ReceiveServer(threading.Thread):
    MAX_FILENAME_LEN = 255
    MAX_FILE_SIZE = 100 * 1024 * 1024
    ALLOWED_EXTENSIONS = {'.txt'}

    def __init__(self, host=None, port=None, received_dir=None, max_workers=10, on_event=None):
        super().__init__(daemon=True)
        self.host = host or config.get('network.server_host', '0.0.0.0')
        self.port = port or config.get('network.server_port', 50000)

        output_dir = Path(config.get('output_dir', 'out'))
        received_subdir = config.get('received_subdir', 'received')
        self.received_dir = (received_dir or output_dir / received_subdir)

        self._stop_event = threading.Event()
        self._executor = ThreadPoolExecutor(max_workers=max_workers, thread_name_prefix="ServerWorker")
        self._observers = []          # liste de callbacks
        if on_event:
            self._observers.append(on_event)

    def add_observer(self, callback):
        if callback not in self._observers:
            self._observers.append(callback)

    def remove_observer(self, callback):
        if callback in self._observers:
            self._observers.remove(callback)

    def _notify(self, event_type, data=None):
        for cb in self._observers:
            try:
                cb(event_type, data)
            except Exception as e:
                logger.error(f"Erreur dans un observateur : {e}")

    def run(self):
        self.received_dir.mkdir(parents=True, exist_ok=True)
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_socket:
            server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
            try:
                server_socket.bind((self.host, self.port))
                server_socket.listen(5)
                server_socket.settimeout(1.0)
                logger.info(f"Serveur démarré sur {self.host}:{self.port}")
                self._notify('started', {'host': self.host, 'port': self.port})
            except Exception as e:
                logger.error(f"Impossible de démarrer le serveur : {e}")
                self._notify('start_failed', {'error': str(e)})
                return
```

```
        while not self._stop_event.is_set():
            try:
                conn, addr = server_socket.accept()
            except socket.timeout:
                continue
            except Exception as e:
                logger.error(f"Erreur accept : {e}")
                break
            self._executor.submit(self._handle_client, conn, addr)

        self._notify('stopped', {})
        logger.info("Serveur arrêté")

def _handle_client(self, conn, addr):
    try:
        # lecture nom
        name_size_bytes = conn.recv(4)
        if not name_size_bytes:
            logger.warning(f"Connexion fermée par {addr} avant le nom")
            self._notify('rejected', {'reason': 'no_name', 'addr': addr})
            return
        name_size = int.from_bytes(name_size_bytes, 'big')
        if name_size > 1024:
            logger.warning(f"Nom trop long ({name_size}) de {addr}")
            self._notify('rejected', {'reason': 'name_too_long', 'addr': addr})
            return

        raw_filename = conn.recv(name_size).decode('utf-8')
        filename = Path(raw_filename).name
        if not filename:
            logger.warning(f"Nom vide de {addr}")
            self._notify('rejected', {'reason': 'empty_name', 'addr': addr})
            return

        filename = re.sub(r'[^a-zA-Z0-9_.-]', '_', filename)
        if len(filename) > self.MAX_FILENAME_LEN:
            filename = filename[:self.MAX_FILENAME_LEN]

        ext = Path(filename).suffix.lower()
        if ext not in self.ALLOWED_EXTENSIONS:
            logger.warning(f"Extension non autorisée '{ext}' de {addr}")
            self._notify('rejected', {'reason': 'extension_not_allowed', 'filename': filename, 'addr': addr})
            return

        # taille des données
        data_size_bytes = conn.recv(8)
        if not data_size_bytes:
            logger.warning(f"Connexion fermée par {addr} avant la taille")
            self._notify('rejected', {'reason': 'no_size', 'addr': addr})
            return
        data_size = int.from_bytes(data_size_bytes, 'big')
        if data_size > self.MAX_FILE_SIZE:
            logger.warning(f"Fichier trop gros ({data_size}) de {addr}")
            self._notify('rejected', {'reason': 'file_too_large', 'size': data_size, 'addr': addr})
```

```
                return

            total_to_read = data_size + 32
            data = b''
            while len(data) < total_to_read:
                chunk = conn.recv(min(4096, total_to_read - len(data)))
                if not chunk:
                    break
                data += chunk

            if len(data) != total_to_read:
                logger.error(f"Taille reçue incorrecte pour {filename} de {addr}")
                self._notify('rejected', {'reason': 'incomplete', 'filename': filename, 'addr': addr})
                return

            file_data = data[:data_size]
            received_hash = data[data_size:]
            computed_hash = sha256(file_data).digest()
            if computed_hash != received_hash:
                logger.error(f"Hash incorrect pour {filename} de {addr}")
                self._notify('rejected', {'reason': 'hash_mismatch', 'filename': filename, 'addr': addr})
                return

            timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
            safe_name = f"{timestamp}_{filename}"
            filepath = self.received_dir / safe_name
            with open(filepath, 'wb') as f:
                f.write(file_data)

            logger.info(f"Fichier reçu de {addr}: {filepath} ({data_size} octets)")
            self._notify('file_received', {
                'filename': filename,
                'path': str(filepath),
                'size': data_size,
                'addr': addr
            })

        except Exception as e:
            logger.error(f"Erreur inattendue avec {addr} : {e}")
            self._notify('rejected', {'reason': 'exception', 'error': str(e), 'addr': addr})
        finally:
            conn.close()

    def stop(self):
        logger.info("Arrêt du serveur demandé")
        self._stop_event.set()
        self._executor.shutdown(wait=True, cancel_futures=False)
        self._notify('stopped', {})
        logger.info("Serveur arrêté")

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\network\utils.py
```

Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\network\utils.py
---
```
# src/network/utils.py
import socket

def get_local_ip() -> str:
    """Retourne l'adresse IP locale utilisée pour les connexions sortantes."""
    try:
        with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
            s.connect(('8.8.8.8', 80))
            return s.getsockname()[0]
    except Exception:
        return '127.0.0.1'
```

--- FIN DU FICHIER ---

==========
FICHIER Python : src\network__init__.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\network__init__.py
---

--- FIN DU FICHIER ---

==========
FICHIER Python : src\services\extraction_service.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\services\extraction_service.py
---
```
# src/services/extraction_service.py
import shutil
from pathlib import Path
from src.extractor.extractor import CodeExtractor
from src.versioning import VersionManager
from src.logger import setup_logger
from src.config import config, ExtractionOptions   # <-- Changement ici

logger = setup_logger(__name__)

class ExtractionService:
    def __init__(self, output_dir=None, version_file=None):
        self.output_dir = Path(output_dir or config.get('output_dir', 'out'))
        self.version_manager = VersionManager(version_file or config.get('version_file', 'extractor_version.txt'))

    def extract_folder(self, folder_path, options, progress_callback=None, log_callback=None, selected_files=None):
        """
        selected_files : liste de chemins relatifs (str) à extraire.
        Si None ou vide, on extrait tous les fichiers trouvés.
        """
        folder_path = Path(folder_path)
        default_options = config.get_all().extraction  # récupère l'objet ExtractionOptions
        # Fusionner avec les options de l'interface (dict)
```

```
    merged_dict = default_options.dict()
    merged_dict.update(options)
    extractor_options = ExtractionOptions(**merged_dict)

    extractor = CodeExtractor(options=extractor_options)

    folder_name = folder_path.name
    self.output_dir.mkdir(parents=True, exist_ok=True)
    next_version = self.version_manager.get_next_version(folder_name, output_dir=self.output_dir)
    output_filename = self.output_dir / f"{folder_name}v{next_version}.txt"

    success, py_count, json_count, txt_count, po_count, mo_count, html_count, css_count, js_count = extractor.extr
        folder_path,
        output_filename,
        progress_callback=progress_callback,
        log_callback=log_callback,
        selected_files=selected_files
    )

    if success:
        self.version_manager.use_version(folder_name, next_version)
        stats = {
            'py_count': py_count,
            'json_count': json_count,
            'txt_count': txt_count,
            'po_count': po_count,
            'mo_count': mo_count,
            'html_count': html_count,
            'css_count': css_count,
            'js_count': js_count,
            'total_files': py_count + json_count + txt_count + po_count + mo_count + html_count + css_count + js_c
            'output_filename': str(output_filename)
        }
        logger.info(f"Extraction réussie : {output_filename}")
        return True, str(output_filename), stats
    else:
        logger.error("Échec de l'extraction")
        return False, None, None

def clean_versions(self, archive=False):
    if archive:
        archive_dir = self.output_dir / config.get('archive_subdir', 'old_out')
        archive_dir.mkdir(parents=True, exist_ok=True)
        moved_files = []
        for item in self.output_dir.iterdir():
            if item.is_file() and item.suffix == '.txt':
                try:
                    shutil.move(str(item), str(archive_dir / item.name))
                    moved_files.append(item.name)
                except Exception as e:
                    logger.error(f"Erreur archivage {item} : {e}")
                    raise
        logger.info(f"Archivés : {', '.join(moved_files)}")
```

```
        self.version_manager.reset()
        logger.info("Historique des versions réinitialisé.")
        return "Nettoyage terminé" + (" (fichiers archivés)" if archive else "")

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\services\pdf_service.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\services\pdf_service.py
--------------------------------------------------------------------------------
# src/services/pdf_service.py
import os
import re
from pathlib import Path
from src.logger import setup_logger

logger = setup_logger(__name__)


class PDFService:
    @staticmethod
    def convert_to_pdf(txt_path: Path, pdf_path: Path):
        """Convertit un fichier texte en PDF en préservant la mise en forme."""
        try:
            from fpdf import FPDF
        except ImportError:
            logger.error("La bibliothèque fpdf n'est pas installée. Installez-la avec 'pip install fpdf'.")
            raise ImportError("fpdf is required for PDF export")

        # Lire le contenu texte
        try:
            with open(txt_path, 'r', encoding='utf-8') as f:
                content = f.read()
        except Exception as e:
            logger.error(f"Erreur lors de la lecture du fichier texte : {e}")
            raise

        # Nettoyer le contenu : remplacer les emojis et caractères non-latin1
        content = PDFService._sanitize_for_pdf(content)

        # Créer le PDF
        pdf = FPDF()
        pdf.set_auto_page_break(auto=True, margin=15)
        pdf.add_page()

        # Utiliser une police qui supporte plus de caractères
        # Tenter d'utiliser une police Unicode si disponible
        try:
            # Essayer d'utiliser fpdf2 avec une police Unicode
            from fpdf import FPDF

            # Vérifier si la version supporte les polices Unicode
            if hasattr(pdf, 'add_font') and PDFService._is_fpdf2():
```

```
            # FPDF2 (plus récent) supporte mieux Unicode
            pdf.set_font('Courier', size=8)
        else:
            # FPDF classique : utiliser une police standard
            pdf.set_font('Courier', size=8)
    except Exception:
        # Fallback
        pdf.set_font('Courier', size=8)

    # Découper en lignes et ajouter au PDF avec gestion des lignes trop longues
    lines = content.splitlines()
    for line in lines:
        # Tronquer les lignes trop longues pour éviter les débordements
        # La largeur de page est de 210mm, la marge à 10mm => 190mm disponibles
        # En police Courier 8, environ 6.5 caractères par mm => ~1235 caractères max
        if len(line) > 1200:
            # Découper la ligne en plusieurs morceaux
            chunks = [line[i:i+1200] for i in range(0, len(line), 1200)]
            for chunk in chunks:
                pdf.cell(0, 5, txt=chunk, ln=True)
        else:
            try:
                pdf.cell(0, 5, txt=line, ln=True)
            except UnicodeEncodeError:
                # Si une ligne pose encore problème, on la nettoie plus agressivement
                safe_line = PDFService._remove_non_latin1(line)
                pdf.cell(0, 5, txt=safe_line, ln=True)

    pdf.output(str(pdf_path))
    logger.info(f"PDF généré : {pdf_path}")
    return pdf_path

@staticmethod
def _sanitize_for_pdf(content: str) -> str:
    """Remplace les emojis et caractères problématiques par des équivalents texte."""
    # Mapping des emojis Unicode vers des équivalents texte
    emoji_map = {
        '[DIR] ': '[DIR] ',
        '[FILE] ': '[FILE] ',
        '[PY] ': '[PY] ',
        '[PO] ': '[PO] ',
        '[MO] ': '[MO] ',
        '[TXT] ': '[TXT] ',
        '[HTML] ': '[HTML] ',
        '[CSS] ': '[CSS] ',
        '[JS] ': '[JS] ',
        '??? ': '???  ',
        '??? ': '???  ',
        '|': '|',
        '[OK] ': '[OK] ',
        '[ERR] ': '[ERR] ',
        '[WARN] ': '[WARN] ',
        '[SRCH] ': '[SRCH] ',
        '[OK] ': '[OK] ',
```

```
        '[ERR] ': '[ERR] ',
        '->': '->',
        '?': '?',
        '?': '?',
        '?': '?',
    }

    # Remplacer les emojis
    for emoji, replacement in emoji_map.items():
        content = content.replace(emoji, replacement)

    # Supprimer les autres caractères non-latin1
    return PDFService._remove_non_latin1(content)

@staticmethod
def _remove_non_latin1(text: str) -> str:
    """Supprime les caractères qui ne sont pas encodables en latin-1."""
    # Définir l'ensemble des caractères valides en latin-1
    # (0x20 à 0x7E sont ASCII imprimables, 0xA0 à 0xFF sont étendus latin-1)
    def is_latin1_char(char):
        code = ord(char)
        # Lettres, chiffres, ponctuation ASCII
        if 0x20 <= code <= 0x7E:
            return True
        # Caractères étendus latin-1 (valides)
        if 0xA0 <= code <= 0xFF:
            return True
        # Tabulation et sauts de ligne
        if code in (0x09, 0x0A, 0x0D):
            return True
        return False

    # Remplacer les caractères invalides
    result = []
    for char in text:
        if is_latin1_char(char):
            result.append(char)
        else:
            # Remplacer par un caractère proche si possible
            replacement = PDFService._get_ascii_equivalent(char)
            if replacement:
                result.append(replacement)
            else:
                # Sinon, remplacer par '?'
                result.append('?')

    return ''.join(result)

@staticmethod
def _get_ascii_equivalent(char: str) -> str:
    """Retourne un équivalent ASCII pour un caractère Unicode si possible."""
    equivalents = {
        'é': 'e', 'è': 'e', 'ê': 'e', 'ë': 'e',
        'à': 'a', 'â': 'a', 'ä': 'a',
```

```
            'î': 'i', 'ï': 'i',
            'ô': 'o', 'ö': 'o',
            'ù': 'u', 'û': 'u', 'ü': 'u',
            'ç': 'c',
            'ñ': 'n',
            'oe': 'oe', 'æ': 'ae',
            '«': '"', '»': '"', '"': '"', '"': '"',
            ''': "'", ''': "'",
            '-': '-', '-': '-',
            '...': '...',
        }
        return equivalents.get(char, None)

    @staticmethod
    def _is_fpdf2() -> bool:
        """Vérifie si fpdf2 est installé (version plus récente)."""
        try:
            from fpdf import FPDF
            # fpdf2 a la classe FPDF avec des fonctionnalités Unicode améliorées
            return hasattr(FPDF, 'add_font')
        except ImportError:
            return False

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\services\version_service.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\services\version_service.py
--------------------------------------------------------------------------------
# src/services/version_service.py
import re
import shutil
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, List, Optional, Tuple

from src.config import config
from src.logger import setup_logger
from src.versioning import VersionManager

logger = setup_logger(__name__)

# Regex pour extraire projet et version d'un nom de fichier
VERSION_FILE_PATTERN = re.compile(r'^(.+?)v(\d+)\.txt$')


@dataclass
class VersionEntry:
    """Métadonnées d'une version d'export."""
    project: str
    version: int
    date: str          # date extraite de l'en-tête
    size: int          # taille en octets
```

```
    file_count: int    # nombre total de fichiers extraits
    line_count: int    # nombre total de lignes
    status: str        # 'active' ou 'archived'
    path: Path         # chemin complet du fichier


class VersionArchiveService:
    """Service de gestion des versions d'export."""

    def __init__(
        self,
        output_dir: Optional[Path] = None,
        archive_subdir: Optional[str] = None,
        version_manager: Optional[VersionManager] = None,
    ):
        self.output_dir = Path(output_dir or config.get('output_dir', 'out'))
        self.archive_subdir = archive_subdir or config.get('archive_subdir', 'old_out')
        self.archive_dir = self.output_dir / self.archive_subdir
        self.version_manager = version_manager or VersionManager(
            config.get('version_file', 'extractor_version.txt')
        )

    def scan_projects(self) -> Dict[str, List[VersionEntry]]:
        """
        Scanne les répertoires de sortie et d'archive pour construire
        la liste des versions par projet.
        Retourne un dictionnaire {nom_projet: [VersionEntry, ...]}.
        """
        projects: Dict[str, List[VersionEntry]] = {}

        # Parcours récursif des deux dossiers
        for root_dir in [self.output_dir, self.archive_dir]:
            if not root_dir.exists():
                continue
            for file_path in root_dir.rglob('*.txt'):
                # Ignorer les fichiers dans des sous-dossiers d'archive ?
                # On veut tous les fichiers .txt
                match = VERSION_FILE_PATTERN.match(file_path.name)
                if not match:
                    continue
                project = match.group(1)
                version = int(match.group(2))
                # Déterminer le statut : si le fichier est dans archive_dir (ou un sous-dossier)
                # mais attention: un fichier archivé peut être dans archive_dir/projet/ ou directement à la racine
                # On considère archivé si le chemin parent contient archive_dir
                if self.archive_dir in file_path.parents or file_path.parent == self.archive_dir:
                    status = 'archived'
                else:
                    status = 'active'

                # Métadonnées (lecture du fichier)
                meta = self._parse_metadata(file_path)
                entry = VersionEntry(
                    project=project,
```

```
                version=version,
                date=meta.get('date', ''),
                size=file_path.stat().st_size,
                file_count=meta.get('file_count', 0),
                line_count=meta.get('line_count', 0),
                status=status,
                path=file_path,
            )
            projects.setdefault(project, []).append(entry)

    # Trier les versions par numéro décroissant pour chaque projet
    for project in projects:
        projects[project].sort(key=lambda e: e.version, reverse=True)

    return projects

@staticmethod
def _parse_metadata(file_path: Path) -> Dict[str, int | str]:
    """
    Extrait la date, le nombre total de fichiers et le nombre total de lignes
    depuis le fichier d'export.
    Retourne un dict avec les clés 'date', 'file_count', 'line_count'.
    """
    result = {'date': '', 'file_count': 0, 'line_count': 0}
    try:
        with open(file_path, 'r', encoding='utf-8') as f:
            content = f.read(4096)  # lire les premiers 4K pour l'en-tête et les stats
    except Exception as e:
        logger.warning(f"Impossible de lire {file_path} pour les métadonnées : {e}")
        return result

    # Extraction de la date
    date_match = re.search(r'Date d\'extraction\s*:\s*(.+)', content)
    if date_match:
        result['date'] = date_match.group(1).strip()

    # Extraction du nombre total de fichiers
    file_match = re.search(r'Nombre total de fichiers\s*:\s*(\d+)', content)
    if file_match:
        result['file_count'] = int(file_match.group(1))

    # Extraction du nombre total de lignes
    line_match = re.search(r'Nombre total de lignes\s*:\s*(\d+)', content)
    if line_match:
        result['line_count'] = int(line_match.group(1))

    return result

def archive(self, entry: VersionEntry) -> Path:
    """
    Archive une version active : déplace le fichier vers archive_dir/project/.
    Retourne le nouveau chemin.
    Lève une exception si le fichier n'existe pas ou si la destination existe.
    """
```

```
    if entry.status != 'active':
        raise ValueError(f"Seules les versions actives peuvent être archivées : {entry.path}")

    dest_dir = self.archive_dir / entry.project
    dest_dir.mkdir(parents=True, exist_ok=True)
    dest_path = dest_dir / entry.path.name

    # Gérer la collision : ajouter un suffixe _conflict si le fichier existe déjà
    if dest_path.exists():
        base = dest_path.stem
        ext = dest_path.suffix
        counter = 1
        while dest_path.exists():
            dest_path = dest_dir / f"{base}_conflict{counter}{ext}"
            counter += 1

    shutil.move(str(entry.path), str(dest_path))
    logger.info(f"Version archivée : {entry.path} -> {dest_path}")
    return dest_path

def restore(self, entry: VersionEntry) -> Path:
    """
    Restaure une version archivée : déplace le fichier vers output_dir/.
    Retourne le nouveau chemin.
    Lève une exception si le fichier n'existe pas ou si la destination existe.
    """
    if entry.status != 'archived':
        raise ValueError(f"Seules les versions archivées peuvent être restaurées : {entry.path}")

    dest_path = self.output_dir / entry.path.name

    # Gérer la collision : ajouter _restored si le fichier existe déjà
    if dest_path.exists():
        base = dest_path.stem
        ext = dest_path.suffix
        # Vérifier si le fichier existe déjà en ajoutant _restored
        dest_path = self.output_dir / f"{base}_restored{ext}"
        if dest_path.exists():
            # Si _restored existe aussi, ajouter un compteur
            counter = 1
            while dest_path.exists():
                dest_path = self.output_dir / f"{base}_restored{counter}{ext}"
                counter += 1

    shutil.move(str(entry.path), str(dest_path))
    logger.info(f"Version restaurée : {entry.path} -> {dest_path}")
    return dest_path

def delete(self, entry: VersionEntry) -> None:
    """Supprime définitivement le fichier de version."""
    if not entry.path.exists():
        logger.warning(f"Le fichier {entry.path} n'existe plus, suppression ignorée.")
        return
    entry.path.unlink()
```

```
        logger.info(f"Version supprimée : {entry.path}")

    def reset_project(self, project_name: str) -> None:
        """
        Réinitialise le compteur de version pour un projet donné.
        """
        self.version_manager.reset_project(project_name)
        logger.info(f"Compteur réinitialisé pour le projet '{project_name}'")

    def reset_all(self) -> None:
        """Réinitialise tous les compteurs."""
        self.version_manager.reset()
        logger.info("Tous les compteurs réinitialisés.")

--- FIN DU FICHIER ---

================================================================================
FICHIER Python : src\services\__init__.py
Type: Python
Chemin complet: C:\dossier tlf\New folder\text exporter\src\services\__init__.py
--------------------------------------------------------------------------------
# src/services/__init__.py

--- FIN DU FICHIER ---

--- FIN DES FICHIERS ---


================================================================================
STATISTIQUES
================================================================================
Nombre de dossiers          : 31
Dossiers 'Python package'    : 11
Fichiers Python (.py)        : 64
Fichiers JSON (.json)        : 2
Fichiers Texte (.txt)        : 0
Fichiers PO (.po)            : 2
Fichiers MO (.mo)            : 2
Fichiers HTML (.html/.htm)   : 0
Fichiers CSS (.css)          : 0
Fichiers JavaScript (.js)    : 0
Nombre total de fichiers     : 70
Nombre de lignes (Python)    : 4472
Nombre de lignes (JSON)      : 39
Nombre de lignes (Texte)     : 0
Nombre de lignes (PO)        : 334
Nombre de lignes (MO)        : 2
Nombre de lignes (HTML)      : 0
Nombre de lignes (CSS)       : 0
Nombre de lignes (JS)        : 0
Nombre total de lignes       : 4847
Volume total extrait         : 195.72 Ko
```